

Trabajo Fin de Grado

Grado en Ingeniería de las Tecnologías de Telecomunicación

Algoritmo de inteligencia artificial explicada para la detección del carcinoma basocelular

Autor: Jose Carlos Navarro García

Tutoras: Begoña Acha Piñero, María del Carmen
Serrano Gotarredona

Dpto. de Teoría de la Señal y Comunicaciones
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2026



Trabajo Fin de Grado
Grado en Ingeniería de las Tecnologías de Telecomunicación

Algoritmo de inteligencia artificial explicada para la detección del carcinoma basocelular

Autor:

Jose Carlos Navarro García

Tutoras:

Begoña Acha Piñero, María del Carmen Serrano Gotarredona

Dpto. de Teoría de la Señal y Comunicaciones
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2026

Agradecimientos

Quiero dar las gracias a mi familia y amigos, y más especialmente a mis padres y mis hermanas, por apoyarme siempre en este largo camino, que tanto esfuerzo y tiempo me ha costado, pero que sin la ayuda y los ánimos de ellos, en todos los malos momentos de ésta larga travesía, hubiera sido imposible de finalizar. Ha sido una etapa dura, pero muy bonita, llena de aprendizaje y que, con mucha resiliencia, hoy termina con éxito.

También me gustaría agradecer a mis tutoras, Carmen y Begoña, por todo lo que he aprendido con ellas, y además, por su magnífico trato personal y cercano, y su comprensión y empatía conmigo.

Jose Carlos Navarro García

Sevilla, 2026

Resumen

El carcinoma basocelular (BCC) es el tipo de cáncer de piel más frecuente, y su diagnóstico precoz depende en gran medida de que el dermatólogo reconozca ciertos patrones dermatoscópicos en la lesión. Los modelos de aprendizaje profundo pueden automatizar esta clasificación, pero su funcionamiento como "caja negra" dificulta que un profesional clínico confíe en el diagnóstico sin saber en qué se basa. Este Trabajo Fin de Grado aborda ese problema: no basta con clasificar bien la lesión, hace falta poder explicar la decisión de forma coherente y visual.

El objetivo es entrenar una red neuronal convolucional (CNN), capaz de detectar, de forma simultánea, varios patrones dermatoscópicos asociados al BCC, y posteriormente aplicar la técnica Grad-CAM para generar mapas de activación que muestren qué zona de la imagen lleva al modelo a detectar cada patrón, comprobando si coincide con la estructura clínica real. Para ello se ha empleado transferencia de aprendizaje con la red VGG16 preentrenada en ImageNet, con una cabeza de clasificación multietiqueta, ya que los patrones no son excluyentes entre sí. Los resultados del entrenamiento se han evaluado con métricas de sensibilidad y especificidad por patrón y posteriormente se han realizado pruebas con un conjunto independiente de fotografías clínicas reales tanto BCC, como de otros tipos de lesiones.

El análisis de los mapas de activación ha permitido identificar limitaciones concretas: la resolución espacial insuficiente de la última capa convolucional de VGG16, el desequilibrio entre patrones en los datos de entrenamiento, y casos en los que el modelo se apoya en elementos ajenos a la piel en lugar de en la estructura dermatoscópica real. Estos resultados muestran que la interpretabilidad no constituye únicamente un complemento visual, sino una herramienta útil para auditar el comportamiento del modelo y detectar errores que las métricas por sí solas no permiten identificar.

Abstract

Basal cell carcinoma (BCC) is the most common type of skin cancer, and its early diagnosis depends largely on the dermatologist recognising certain dermatoscopic patterns in the lesion. Deep learning models can automate this classification, but their ‘black box’ nature makes it difficult for a clinician to trust the diagnosis without knowing the reasoning behind it. This final-year project addresses this problem: it is not enough simply to classify the lesion correctly; it is also necessary to be able to explain the decision in a coherent and visual manner.

The aim is to train a convolutional neural network (CNN) capable of simultaneously detecting several dermatoscopic patterns associated with BCC, and subsequently apply the Grad-CAM technique to generate activation maps showing which area of the image leads the model to detect each pattern, verifying whether this corresponds to the actual clinical structure. To achieve this, transfer learning was employed using the VGG16 network pre-trained on ImageNet, with a multi-label classification head, as the patterns are not mutually exclusive. The training results were evaluated using sensitivity and specificity metrics per pattern, and subsequent testing was carried out on an independent set of real clinical photographs of both BCC and other types of lesions.

Analysis of the activation maps has enabled the identification of specific limitations: the insufficient spatial resolution of the final convolutional layer of VGG16, the imbalance between patterns in the training data, and instances where the model relies on elements unrelated to the skin rather than on the actual dermatoscopic structure. These results show that interpretability is not merely a visual complement, but a useful tool for auditing the model’s behaviour and detecting errors that metrics alone cannot identify.

Índice

Agradecimientos	I
Resumen	III
Abstract	V
1 Introducción	1
1.1 Motivación del Proyecto	1
1.2 El papel de la Inteligencia Artificial Explicable (XAI)	1
1.3 Objetivos	2
2 Estado del Arte	3
2.1 Deep Learning y CNNs en la Imagen Médica	3
2.2 Sistemas de Ayuda al Diagnóstico del Carcinoma Basocelular (BCC)	4
2.3 Mapas de activación e Inteligencia Artificial Explicable (XAI)	4
3 Diagnóstico médico: Patrones BCC	7
3.1 La dermatoscopia como técnica diagnóstica	7
3.2 El método diagnóstico en dos etapas	7
3.3 Criterios dermatoscópicos del carcinoma basocelular	8
4 Descripción de la red neuronal utilizada	11
4.1 Transferencia de Aprendizaje	11
4.2 Arquitectura del modelo: VGG16	11
4.3 Preprocesado de la entrada y configuración del entrenamiento	14
4.4 Capas implementadas: la cabeza de clasificación	15
4.5 Función de pérdida y optimizador	16
5 Mapas de activación (Grad-CAM)	17
5.1 Fundamento teórico de Grad-CAM	17
5.2 Implementación del algoritmo Grad-CAM en Python	18
6 Resultados	21
6.1 Metodología de evaluación	21
6.2 Conjunto de Datos Utilizado	21
6.3 Métricas estadísticas de evaluación	22
6.3.1 Exactitud Global (<i>Accuracy</i>)	23
6.3.2 Sensibilidad (<i>Sensitivity</i>)	23

6.3.3	Especificidad (<i>Specificity</i>)	23
6.3.4	Valor Predictivo Positivo (<i>Precision</i>)	23
6.3.5	F1-score	24
6.3.6	Matriz de confusión	24
6.4	Evolución del entrenamiento	24
6.5	Resultados de clasificación por patrón (Test interno)	25
6.6	Clasificación clínica global (BCC frente a No-BCC)	25
6.7	Casos de estudio: Análisis visual con Grad-CAM sobre el conjunto BCC/No-BCC	27
6.7.1	Carcinomas basocelulares con patrón correctamente identificado	27
6.7.2	Carcinoma basocelular con activación errónea del patrón reticular	30
6.7.3	Lesiones No-BCC con estructuras visualmente similares a patrones de BCC	30
6.7.4	Lesión No-BCC correctamente descartada mediante el patrón reticular	33
7	Conclusiones y líneas futuras	35
	Índice de Figuras	37
	Índice de Tablas	39
	Índice de Códigos	41
	Bibliografía	43
A	Anexo: Códigos	47
A.1	Configuración inicial	47
A.2	Parte A: entrenamiento del modelo	49
A.3	Parte B: evaluación del modelo	52
A.4	Parte C: mapas de activación (Grad-CAM)	55

1 Introducción

1.1. Motivación del Proyecto

El cáncer de piel es el tipo de cáncer más común a nivel mundial [1]. Dentro de esta categoría, los tumores no melanocíticos son muy frecuentes, destacando el carcinoma basocelular (BCC), que representa el 75 % de todos los cánceres de piel. Actualmente, la detección de este cáncer se realiza mediante la inspección visual por parte de un dermatólogo experimentado. Sin embargo, existen múltiples lesiones benignas que pueden confundirse fácilmente con este tipo de cáncer, lo que a menudo conduce a la realización de biopsias innecesarias [2].

Adicionalmente, la alta incidencia de este cáncer cutáneo ha provocado un aumento significativo en la cantidad de imágenes recibidas en los centros hospitalarios a través de teledermatología, lo que genera una gran sobrecarga de trabajo para los dermatólogos. Por tanto, resulta crucial el desarrollo de herramientas de detección automática que agilicen la atención al paciente y mejoren la eficiencia del diagnóstico clínico.

La teledermatología ofrece a los médicos generalistas la posibilidad de recabar la opinión de un dermatólogo especialista sobre las lesiones cutáneas que consideren sospechosas, sin necesidad de derivar a los pacientes. Las consultas de teledermatología pueden ser de tipo «almacenamiento y reenvío», en las que se envían imágenes digitales de una lesión a un dermatólogo para una posterior revisión, o bien consultas en directo e interactivas mediante videoconferencia que conectan al paciente, al médico remitente y al dermatólogo en tiempo real.[3]

1.2. El papel de la Inteligencia Artificial Explicable (XAI)

En los últimos años, el uso de la Inteligencia Artificial (IA) y las redes neuronales profundas para la clasificación de imágenes ha experimentado un enorme crecimiento. No obstante, la principal preocupación respecto a la implementación de estas herramientas de diagnóstico asistido por ordenador (CAD) en medicina, es la falta de explicabilidad de sus clasificaciones. Las redes neuronales suelen considerarse como "cajas negras" que proporcionan una etiqueta final para cada clase, pero sin revelar las decisiones internas que han llevado a dicho resultado.

Para que estas herramientas sean fiables en el entorno clínico, es necesario un enfoque de clasificación explicable. Esto implica que la detección no se limite a una clasificación binaria o porcentual, sino que identifique visualmente los mismos patrones dermatoscópicos que los médicos emplean para establecer su diagnóstico. Muchos enfoques actuales de IA médica se centran solamente en la precisión del diagnóstico, prestando muy poca atención a la interpretabilidad del modelo utilizado.

1.3. Objetivos

Los objetivos principales de este Trabajo Fin de Grado son:

- Implementar un algoritmo de Inteligencia Artificial capaz de clasificar imágenes dermatoscópicas en BCC y no-BCC.
- Superar la limitación de los modelos de "caja negra" desarrollando una metodología de IA Explicable (XAI) basada en características clínicas.
- Generar mapas de activación utilizando Grad-CAM, que localicen e identifiquen visualmente los patrones dermatoscópicos específicos de la lesión, sirviendo de apoyo y priorización para el dermatólogo, y que permitan detectar errores en la clasificación.

2 Estado del Arte

2.1. Deep Learning y CNNs en la Imagen Médica

Históricamente, la clasificación automática de lesiones de piel se basaba en la extracción manual de características clínicas. Sin embargo, con la llegada de las redes neuronales convolucionales (CNN), el paradigma ha ido cambiando hacia la extracción automática de características.

El uso de CNNs en el análisis de imágenes médicas ha ido creciendo paulatinamente desde que estas arquitecturas demostraron ser superiores frente a otras técnicas de aprendizaje automático, aplicándose desde entonces a tareas tan distintas como la segmentación de estructuras para planificar radioterapia, la detección de lesiones en radiografía de tórax o el pronóstico de pacientes a partir de características cuantitativas extraídas automáticamente de una imagen [4].

Dentro de la clasificación de lesiones cutáneas mediante CNN, el trabajo de Esteva *et al.* es de los más citados: su modelo alcanzó un rendimiento comparable al de un dermatólogo certificado clasificando distintos tipos de cáncer de piel a partir de fotografías clínicas [5].

¿De dónde viene realmente la mejora al usar una red preentrenada? Tammina lo comprueba enfrentando tres aproximaciones sucesivas sobre un mismo problema de clasificación binaria con pocas imágenes: una CNN entrenada desde cero, la misma CNN con aumento de datos (*data augmentation*), y VGG16 preentrenada en ImageNet usada como extractor de características. Las precisiones de validación obtenidas fueron 72.40 %, 79.20 % y 95.40 % respectivamente, y dejan bastante claro que la mejora que se ha producido no viene tanto del aumento artificial de datos como del uso de la transferencia de aprendizaje [6].

La arquitectura VGG16 no es la más eficiente, ni la de mayor precisión en todos los campos. Tyagi *et al.* la compararon frente a Inception v3 y VGG19 en un estudio de identificación de enfermedades en hojas de plantas, donde se quedó en un 63.70 % de precisión frente al 70.12 % de Inception v3, aunque por delante del 57.28 % de VGG19 [7]. A pesar de obtener ese resultado, los autores destacan que VGG16 es menos propensa a que se produzca sobreajuste y resulta muy adecuada cuando disponemos de conjuntos de datos pequeños o de recursos limitados de computación, que son condiciones que se presentan en este TFG, con lo que se da mas peso a la idea de que hemos escogido una arquitectura útil para nuestro trabajo [7].

Sheriff *et al.* emplearon también VGG16 con éxito, para la detección temprana de cáncer de pulmón, combinándola con aumento de datos y el optimizador Adam para extraer características complejas y alcanzar unos resultados y métricas de evaluación muy satisfactorios. [8].

La biometría en tiempo real es otro campo que nos da una pista sobre por qué VGG16 encaja en dermatoscopia. Vimal y Shirivastava lograron detectar y clasificar rostros con un 93 % de precisión incluso con fondos muy ruidosos y variaciones de escala [9]; esa capacidad para discriminar el objeto de interés frente a ruido espacial es la que necesita un clasificador dermatoscópico para ignorar posibles artefactos como el vello corporal o el marco oscuro de la lente dermatoscópica, y concentrarse únicamente en las estructuras patológicas del BCC [9].

2.2. Sistemas de Ayuda al Diagnóstico del Carcinoma Basocelular (BCC)

El presente trabajo que nos ocupa se centra en la detección de los patrones dermatoscópicos característicos de un tipo concreto de cáncer de piel, el carcinoma basocelular (BCC). En el estudio de Serrano *et al.* en el que se inspira este TFG, se demuestra que es posible mejorar el rendimiento de estas redes neuronales si son capaces de guiar a la red para que preste atención a estructuras específicas del BCC [2].

No todos los antecedentes son igual de optimistas. Marka *et al.* revisaron 39 estudios de clasificación automática de cáncer de piel no melanoma y, aunque confirmaron la eficacia diagnóstica de las redes neuronales frente a los clasificadores tradicionales, señalaron una limitación recurrente: la mayoría de las aproximaciones dependían de la delimitación manual de bordes o de características extraídas a mano, sin arquitecturas automatizadas del todo y explicables clínicamente[10].

La detección de estructuras morfológicas individuales también se ha abordado con aprendizaje no supervisado. Un ejemplo es la translucidez, un rasgo con una especificidad clínica del 93 % en el diagnóstico del BCC [11]. Huang *et al.* la detectaron analizando parches de fotografías clínicas macroscópicas, evitando así la deformación óptica que introduce el contacto de la lente, mediante un autoencoder disperso apilado (SSAE) acoplado a un clasificador Softmax, con el que alcanzaron ese mismo 93 % de precisión [11].

Cheng *et al.* localizaron las telangiectasas mediante una técnica de caída de color local (*local color drop*) combinada con un filtrado de ruido, y extrayendo después distintas características de forma, tamaño y recuento a partir de la máscara resultante para alimentar un clasificador de redes neuronales. Sobre 59 carcinomas basocelulares y 152 lesiones benignas, el sistema alcanzó una precisión diagnóstica del 96,7 % [12]. Esta cifra nos confirma que segmentar y normalizar espacialmente los patrones vasculares del BCC permite distinguirlo de los vasos sanguíneos de la piel que tienen un daño solar [12].

Un enfoque similar de segmentación se aplicó a los nidos ovoides azul-grisáceos, estructuras que por sus variaciones de tamaño y color solemos confundir fácilmente con elementos de lesiones benignas. Guvenç *et al.* propusieron un método de segmentación por expansión de sectores a partir de un bloque semilla, refinando después los bordes que eran irregulares mediante elipses que se ajustaban mejor. Tras extraer 21 características de color y una de tamaño, y mediante una regresión logística sobre 68 carcinomas y 131 lesiones benignas, se identificaron la cromaticidad y la varianza del canal azul como los atributos más discriminativos, con una tasa de clasificación del 96,5 % [13].

Kefel *et al.*, por su parte, se centraron en el rubor rosado (*pink blush*) y la semitranslucidez, evaluando métricas de suavidad y brillo en cuatro bandas de color (luminancia, rojo, verde y azul) mediante umbralización dinámica y corrección de vecindad. Validado mediante regresión logística sobre 1024 imágenes, el sistema alcanzó un Área Bajo la Curva (AUC) de 0,877 de forma completamente automática, prueba de que la semitranslucidez patológica puede identificarse solo con estadísticas de textura y color [14].

Por último, Kharazmi *et al.* exploraron la fusión de datos multimodales: un autoencoder disperso (SAE) aprende de forma no supervisada los patrones vasculares de la imagen dermatoscópica, y esos mapas de características se combinan con el perfil clínico del paciente (edad, sexo, tamaño y localización de la lesión) antes de alimentar un clasificador Softmax. Para una cantidad de 1199 imágenes, el sistema alcanzó un AUC del 91,1 %, mostrando que combinar extracción morfológica profunda con metadatos clínicos mejora la robustez del diagnóstico [15].

2.3. Mapas de activación e Inteligencia Artificial Explicable (XAI)

A pesar de la alta precisión alcanzada por los modelos basados en redes neuronales profundas, su adopción en entornos clínicos reales se ve frenada por su condición de "caja negra": los especialistas médicos necesitan comprender en qué características de la imagen se basa el algoritmo para emitir un diagnóstico. Para responder a este problema ha surgido el campo de la Inteligencia Artificial Explicable (XAI), que agrupa

técnicas capaces de localizar qué regiones de la imagen han pesado más en la decisión de la red sin modificar su arquitectura interna; entre los enfoques que se basan en gradientes que generan estos mapas de calor, destaca *Grad-CAM*, la técnica empleada en este trabajo [16].

Esta exigencia no es solo teórica, Ben *et al.* sitúan la integración de técnicas XAI en los sistemas de diagnóstico asistido por ordenador (CAD) como un requisito fundamental para ganar confianza clínica y evitar que la red se apoye en ruido de fondo o artefactos [17].

En tumores cerebrales, por ejemplo, uno de los principales riesgos de los modelos convolucionales es correlacionar sus predicciones con tejido sano circundante o artefactos irrelevantes del fondo de la imagen. Iftikhar *et al.* muestran que inspeccionar el flujo de gradientes de *Grad-CAM* en las capas convolucionales permite auditar este comportamiento interno, comprobando que el aprendizaje esté anclado a estructuras patológicas reales sin comprometer la precisión diagnóstica [18].

Combinar varias técnicas a la vez es otra vía posible. Jumaili y Sonuç determinan la etapa del Alzheimer mediante resonancia magnética integrando simultáneamente *Grad-CAM*, explicaciones locales independientes del modelo (LIME) y mecanismos de atención: mientras *Grad-CAM* ilustra la distribución continua del peso espacial, LIME segmenta de forma explícita las regiones críticas, y entre las dos confirman que el algoritmo se fija en biomarcadores reales, como la atrofia del hipocampo, y no en variables de confusión [19].

La arquitectura elegida también condiciona la calidad de estas explicaciones. Comparando clasificadores oncológicos sobre resonancias magnéticas, Yoon y Lin observan que los modelos más profundos como ResNet50 tienden a capturar un contexto espacial demasiado amplio y diluyen el foco de atención, mientras que VGG16 produce mapas *Grad-CAM* más concentrados y precisos sobre la lesión, lo que es un argumento adicional a favor de la arquitectura empleada en este TFG, y destacan además, que estas técnicas sirven tanto para validar aciertos como para auditar falsos positivos [20].

Más allá de que el algoritmo sea preciso, la utilidad clínica real de la IA depende de su capacidad para cooperar con el especialista. En un estudio pionero con una arquitectura ResNet34, Tschandl *et al.* comprobaron que apoyar el diagnóstico multiclase de lesiones cutáneas, incluido el BCC, con mapas de activación de clases mejora la precisión conjunta por encima del rendimiento aislado del médico o de la máquina, y que son precisamente los dermatólogos con menos experiencia quienes más se benefician de esa explicación visual [21].

3 Diagnóstico médico: Patrones BCC

3.1. La dermatoscopia como técnica diagnóstica

La dermatoscopia es una técnica diagnóstica no invasiva que permite observar estructuras de la epidermis y la dermis que resultan invisibles a simple vista, mediante un sistema óptico de amplificación e iluminación que evita la refracción de la capa córnea (con un filtro de luz polarizada o mediante un medio de inmersión). Aunque sus orígenes se remontan al siglo XVII, no fue hasta el siglo XX cuando alcanzó el nivel de perfeccionamiento y difusión que la convierte hoy en una herramienta imprescindible en la práctica clínica de cualquier dermatólogo [22].

Su utilidad en el diagnóstico del melanoma está firmemente establecida: mientras que la sensibilidad del diagnóstico clínico a simple vista, incluso por dermatólogos con amplia experiencia, es del 70–85 %, el uso de la dermatoscopia en manos expertas incrementa esa sensibilidad hasta el 92 % [22]. Precisamente por la enorme relevancia del melanoma, la mayoría de estudios de dermatoscopia de la segunda mitad del siglo XX se centraron en lesiones melanocíticas; sin embargo, la técnica ha demostrado ser igualmente útil en el diagnóstico de lesiones *no* melanocíticas, entre ellas, el carcinoma basocelular, la lesión de interés en este trabajo fin de grado [22].

3.2. El método diagnóstico en dos etapas

El procedimiento diagnóstico aceptado y empleado por la inmensa mayoría de dermatólogos en todo el mundo es el llamado *método de las dos etapas* [22]:

1. **Primera etapa.** Se determina si la lesión cumple los criterios de una lesión melanocítica (originada en los melanocitos, como un nevo o un melanoma) o, por el contrario, de una lesión no melanocítica.
2. **Segunda etapa.** Si la lesión es melanocítica, se aplica alguno de los métodos específicos para distinguir lesiones malignas de benignas (análisis de patrones, método ABCD de Stolz, método de Menzies, o la regla de los siete puntos de Argenziano). Si la lesión es no melanocítica, se buscan en su lugar los criterios dermatoscópicos específicos de las entidades no melanocíticas más frecuentes: queratosis seborreica, carcinoma basocelular, dermatofibroma y lesiones vasculares.

Este trabajo se sitúa por completo dentro de la segunda rama de la segunda etapa: los siete patrones que nuestra red neuronal va a aprender a reconocer, son, precisamente, los criterios dermatoscópicos específicos que permiten reconocer un carcinoma basocelular frente al resto de lesiones no melanocíticas.

3.3. Criterios dermatoscópicos del carcinoma basocelular

En el año 2000, Menzies y cols. describieron los hallazgos dermatoscópicos característicos del carcinoma basocelular y desarrollaron un método diagnóstico que, con modificaciones, se sigue utilizando actualmente. Las estructuras dermatoscópicas características del carcinoma basocelular son:[23]

- **Múltiples glóbulos azul-gris.** Se trata de estructuras redondeadas, múltiples, de pequeño tamaño y de coloración azul-grisácea.
- **Nidos grandes ovoides azul-gris.** Son áreas ovoides o redondeadas, bien delimitadas, de coloración azul grisácea, que tienen un mayor tamaño que los glóbulos.
- **Áreas en hoja de arce.** Son estructuras pigmentadas (marrones o azul-grisáceas) que forman extensiones bulbosas, habitualmente en la periferia del tumor y que recuerdan la silueta de una hoja.
- **Estructuras en rueda de carro.** Se describen dermatoscópicamente como proyecciones radiales, marrones o azul-grisáceas que convergen en un punto central más hiperpigmentado, recordando los radios de una rueda.
- **Telangiectasias ramificadas.** Se trata de vasos habitualmente bien enfocados que muestran una morfología arborescente (vasos gruesos que se ramifican en vasos más delgados).
- **Ulceración.** Se describen áreas denudadas en la superficie de la lesión que suelen observarse dermatoscópicamente cubiertas por una costra serohemática.

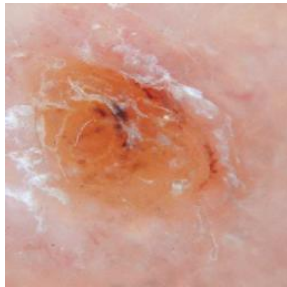
Por otra parte tenemos un patrón que cuando está presente indicaría que no nos encontramos frente a un carcinoma basocelular:

- **Retículo Pigmentado.** Es un patrón formado por líneas pigmentadas (habitualmente de un tono marrón) que se entrecruzan formando una especie de malla o panal de abeja sobre un fondo más claro, cubriendo total o parcialmente la lesión.

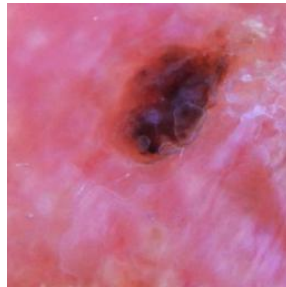
En esta misma línea, Menzies *et al.* describieron un método diagnóstico estructurado para el carcinoma basocelular pigmentado. Según este enfoque, una lesión pigmentada es sugerente de ser un BCC si cumple dos condiciones clínicas fundamentales: la ausencia de retículo pigmentado (un patrón característico de las lesiones melanocíticas) y la presencia de, al menos, uno de los criterios dermatoscópicos descritos anteriormente [23]. La aplicación de este método demostró ser altamente eficaz, alcanzando una sensibilidad del 93 % y una especificidad del 89 % frente al melanoma, así como un 92 % de especificidad respecto a otras lesiones cutáneas benignas pigmentadas [23].

Posteriormente, en el año 2002, Peris *et al.* validaron empíricamente estos criterios y el método diagnóstico en su conjunto, concluyendo que resultan altamente reproducibles y de fácil aplicación en el diagnóstico clínico rutinario [24].

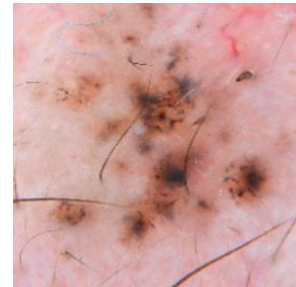
Para comprender la base del diagnóstico clínico que el modelo de Inteligencia Artificial debe replicar, es fundamental definir visualmente las estructuras biológicas de interés. En la Figura 3.1 se ilustran los siete patrones dermatoscópicos principales asociados al carcinoma basocelular. Estas muestras han sido extraídas del conjunto de entrenamiento.



(a) Ulceración



(b) Nidos Ovoides



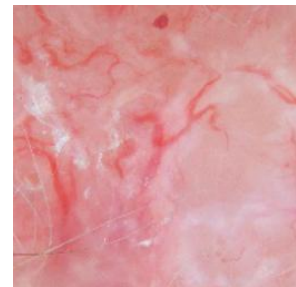
(c) Glóbulos



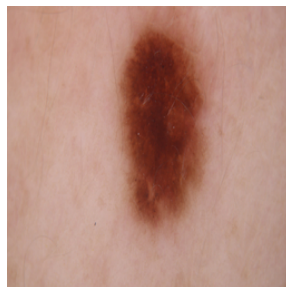
(d) Hojas de Arce



(e) Ruedas de Carro



(f) Telangiectasias



(g) Retículo Pigmentado

Figura 3.1: Ejemplos visuales de los 6 patrones dermatoscópicos característicos del carcinoma basocelular y del patrón negativo retículo pigmentado.

4 Descripción de la red neuronal utilizada

Este capítulo describe la arquitectura de red neuronal empleada para la clasificación de los siete patrones dermatoscópicos, los seis asociados al BCC y un patrón de exclusión (detallados en el Capítulo 3). En el diseño del modelo se distinguen dos componentes: la red convolucional VGG16 con los pesos preentrenados sobre el conjunto de datos ImageNet, de la que se descongelan y reentrenan (*fine-tuning*) únicamente sus últimas capas para adaptarlas al dominio dermatoscópico, mientras que el resto permanece fijo; y una parte implementada y entrenada por completo específicamente para este trabajo, una pequeña red densa (la “cabeza” de clasificación) que aprende a reconocer los siete patrones a partir de las características que produce VGG16.

4.1. Transferencia de Aprendizaje

El entrenamiento de una red convolucional profunda desde cero requiere, por norma general, cientos de miles o millones de imágenes etiquetadas para que el modelo aprenda a extraer filtros útiles que reconozcan bordes, texturas y formas complejas. El conjunto de datos disponible en este trabajo es muy reducido en comparación, por lo que entrenar una red profunda desde su inicio resulta inviable, ya que provocaría casi con total seguridad un sobreajuste (*overfitting*) del modelo.

La solución adoptada, habitual en el ámbito de la visión por computador cuando se dispone de pocos datos, es la *transferencia de aprendizaje* (*transfer learning*). Esta técnica consiste en reutilizar una red previamente entrenada sobre un conjunto de datos masivo y genérico (en este caso, ImageNet, que cuenta con más de un millón de imágenes de mil categorías distintas de objetos cotidianos) para aprovechar los filtros que ya ha consolidado, como bordes, texturas, patrones de color y formas, utilizándolas como un extractor de características genérico para imágenes de dermatoscopia, un dominio completamente distinto al original. De este modo, únicamente se entrena desde cero una pequeña parte añadida al final de la red, diseñada de manera específica para la tarea de clasificación abordada en este proyecto.

Dentro de la transferencia de aprendizaje existen dos variantes principales: usar la red preentrenada como extractor de características completamente fijo, sin modificar ninguno de sus pesos, o aplicar además un *ajuste fino* (*fine-tuning*), descongelando y reentrenando una parte de sus capas para adaptar sus filtros al nuevo dominio. En este trabajo se ha optado por la segunda opción, descongelando las últimas capas de VGG16, tal como se detalla en la siguiente sección.

4.2. Arquitectura del modelo: VGG16

Para abordar este problema de clasificación de imágenes dermatoscópicas, se ha empleado la arquitectura VGG16, una red neuronal convolucional (CNN) ampliamente reconocida por su profundidad y uniformidad.

Esta red se caracteriza por utilizar de forma exclusiva filtros convolucionales de dimensiones 3×3 , lo que permite extraer características muy complejas, como los patrones típicos del BCC, manteniendo un coste computacional bastante manejable.



Figura 4.1: Arquitectura VGG16 [25].

La arquitectura VGG16 [26], es un modelo convolucional compuesto por 16 capas con pesos entrenables (13 capas convolucionales y 3 capas totalmente conectadas en su versión original). Esta red se encuentra disponible directamente a través de la biblioteca Keras, junto con sus pesos preentrenados en ImageNet. Se trata de una arquitectura de diseño uniforme en comparación con redes más modernas, dado que todas sus capas convolucionales utilizan filtros de 3×3 y todas sus capas de *pooling* operan con ventanas de 2×2 . Esta simplicidad estructural facilita enormemente la comprensión del flujo de datos y la posterior interpretación de los mapas de activación generados mediante Grad-CAM. Además, VGG16 es una de las arquitecturas más consolidadas como extractor de características en la literatura científica sobre clasificación de imágenes médicas, siendo la red utilizada en el artículo de referencia que inspira este trabajo [2].

En la Figura 4.2 se muestra un diagrama de bloques de la arquitectura VGG16.

VGG16 se ha cargado con `include_top=False`, lo que elimina las tres capas totalmente conectadas originales de la red (diseñadas para clasificar entre las 1000 categorías de ImageNet) y conserva únicamente su parte convolucional. Esta parte convolucional está organizada en cinco bloques, cada uno formado por dos o tres capas de convolución seguidas de una capa de *max pooling* que reduce a la mitad el tamaño espacial del mapa de activación. Todas las capas convolucionales usan *stride* 1 y relleno *same* (de forma que no reducen el tamaño espacial dentro de un mismo bloque, solo combinan información de píxeles vecinos), y todas las capas de *max pooling* usan *stride* 2 [7]:

- **Bloque 1:** 2 capas Conv2D de 64 filtros (3×3 , activación ReLU) + MaxPooling2D (2×2).
- **Bloque 2:** 2 capas Conv2D de 128 filtros (3×3 , ReLU) + MaxPooling2D (2×2).
- **Bloque 3:** 3 capas Conv2D de 256 filtros (3×3 , ReLU) + MaxPooling2D (2×2).
- **Bloque 4:** 3 capas Conv2D de 512 filtros (3×3 , ReLU) + MaxPooling2D (2×2).
- **Bloque 5:** 3 capas Conv2D de 512 filtros (3×3 , ReLU) + MaxPooling2D (2×2).

En total, 13 capas convolucionales y 5 capas de *max pooling*, con un número creciente de filtros (de 64 a 512) a medida que se avanza en profundidad: Las primeras capas extraen características básicas como bordes

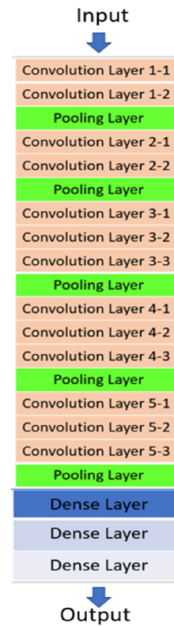


Figura 4.2: Diagrama de bloques de la arquitectura VGG16.

y texturas, mientras que las capas más profundas combinan estas características para representar estructuras mas complejas.

Las imágenes de entrada tienen un tamaño de 256×256 píxeles con 3 canales de color ($\text{IMG_SIZE} = 256$), por lo que la entrada a la red tiene forma $(256, 256, 3)$. Como cada uno de los 5 bloques reduce a la mitad el tamaño espacial mediante su capa de *max pooling*, el mapa de activación de salida de la última capa tiene un tamaño espacial de $256/2^5 = 8$ píxeles por lado, con 512 canales (uno por cada filtro del último bloque): la salida de VGG16 es, por tanto, un tensor de forma $(8, 8, 512)$ por cada imagen de entrada. Este tamaño tan reducido (8×8) es, como se explica en el Capítulo 5, la causa de que los mapas de calor de Grad-CAM tengan una resolución espacial limitada.

De estas 13 capas convolucionales, la gran mayoría permanece congelada: se cargan directamente desde ImageNet (`weights='imagenet'`) y no se modifican durante el entrenamiento. Sin embargo, se descongelan y reentrenan las últimas 5 capas de la red, `block4_pool`, `block5_conv1`, `block5_conv2`, `block5_conv3` y `block5_pool`, permitiendo que sus pesos se ajusten al dominio dermatoscópico mediante *fine-tuning*. Congelar por completo la parte convolucional de una red preentrenada y usarla exclusivamente como extractor de características fijo es una práctica bien documentada cuando se dispone de muy pocos datos [6]; sin embargo, en este trabajo se comprobó experimentalmente que permitir un ajuste fino limitado de las capas más profundas, las que codifican las estructuras más abstractas y específicas del dominio, frente a las primeras capas, que capturan patrones muy genéricos (bordes, texturas, colores) útiles en casi cualquier tipo de imagen, mejora tanto la clasificación final como, sobre todo, la calidad de los mapas de Grad-CAM (Capítulo 5), que pasan a fundamentarse en características realmente adaptadas al problema en lugar de en filtros genéricos de ImageNet.

El número de capas a descongelar se determinó de forma empírica, comparando configuraciones con 5, 8 y 18 capas descongeladas (esta última equivalente a reentrenar toda la parte convolucional de VGG16). Descongelar más capas de las necesarias no mejoró los resultados: dado el tamaño reducido del conjunto de entrenamiento frente a los más de 14 millones de parámetros de la parte convolucional de VGG16, cuantas más capas se permiten entrenar, mayor es el riesgo de sobreajuste y de inestabilidad durante el entrenamiento. La configuración de 5 capas descongeladas ofrece el mejor equilibrio entre adaptación al dominio dermatoscópico y estabilidad del entrenamiento con los datos disponibles, además de tener unos tiempos de

entrenamiento mucho menores que las otras configuraciones, y es la que se ha adoptado como definitiva.

En la Tabla 4.1 se detalla la estructura completa de la red neuronal empleada, mostrando la evolución de la dimensión espacial de los tensores a través de los distintos bloques convolucionales de la arquitectura base (VGG16) y la etapa clasificadora final.

Tabla 4.1: Estructura de capas, dimensiones de salida y parámetros del modelo.

Capa (Nombre)	Tipo	Dimensión de Salida	Parámetros
input_layer_3	InputLayer	(None, 256, 256, 3)	0
block1_conv1	Conv2D	(None, 256, 256, 64)	1.792
block1_conv2	Conv2D	(None, 256, 256, 64)	36.928
block1_pool	MaxPooling2D	(None, 128, 128, 64)	0
block2_conv1	Conv2D	(None, 128, 128, 128)	73.856
block2_conv2	Conv2D	(None, 128, 128, 128)	147.584
block2_pool	MaxPooling2D	(None, 64, 64, 128)	0
block3_conv1	Conv2D	(None, 64, 64, 256)	295.168
block3_conv2	Conv2D	(None, 64, 64, 256)	590.080
block3_conv3	Conv2D	(None, 64, 64, 256)	590.080
block3_pool	MaxPooling2D	(None, 32, 32, 256)	0
block4_conv1	Conv2D	(None, 32, 32, 512)	1.180.160
block4_conv2	Conv2D	(None, 32, 32, 512)	2.359.808
block4_conv3	Conv2D	(None, 32, 32, 512)	2.359.808
block4_pool	MaxPooling2D	(None, 16, 16, 512)	0
block5_conv1	Conv2D	(None, 16, 16, 512)	2.359.808
block5_conv2	Conv2D	(None, 16, 16, 512)	2.359.808
block5_conv3	Conv2D	(None, 16, 16, 512)	2.359.808
block5_pool	MaxPooling2D	(None, 8, 8, 512)	0
global_average_pooling2d_3	GlobalAveragePooling2D	(None, 512)	0
dense_6	Dense	(None, 512)	262.656
dropout_3	Dropout	(None, 512)	0
dense_7	Dense	(None, 7)	3.591

Del total de 14.980.935 parámetros que conforman la red, 7.345.671 han sido configurados como entrenables durante el proceso de optimización, mientras que los 7.635.264 parámetros restantes corresponden a las capas iniciales congeladas de la arquitectura base, aplicando de esta forma los principios de la transferencia de aprendizaje (*Transfer Learning*).

4.3. Preprocesado de la entrada y configuración del entrenamiento

Antes de pasar cada imagen por VGG16 es imprescindible aplicarle el mismo preprocesado con el que se entrenó originalmente sobre ImageNet, mediante la función `{tf.keras.applications.vgg16.preprocess\_input}`: esta función convierte la imagen de RGB a BGR y resta a cada canal de color su valor medio calculado sobre el conjunto de entrenamiento de ImageNet. Sin este paso las características que produce VGG16 no son válidas, porque la red espera ver las imágenes codificadas exactamente igual que durante su entrenamiento original.

Una vez preprocesadas las imágenes, el modelo completo se entrena de principio a fin durante `EPOCHS = 20` épocas, con un tamaño de lote (*batch size*) de 32 imágenes.

Código 4.1: Configuración de las épocas de entrenamiento y ajuste del modelo completo.

```
# VGG16 espera las imagenes preprocesadas igual que en ImageNet: conversion a BGR
y
```

```
# resta de la media de cada canal.
train_set_x_prep = tf.keras.applications.vgg16.preprocess_input(train_set_x.astype('float32'))
val_set_x_prep = tf.keras.applications.vgg16.preprocess_input(val_set_x.astype('float32'))
test_set_x_prep = tf.keras.applications.vgg16.preprocess_input(test_set_x.astype('float32'))

EPOCHS = 20
BATCH_SIZE = 32

historia = modelo_completo.fit(
    x=train_set_x_prep, y=train_set_y,
    epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1,
    validation_data=(val_set_x_prep, val_set_y),
    shuffle=True,
)
```

4.4. Capas implementadas: la cabeza de clasificación

La cabeza de clasificación se ha diseñado e implementado específicamente para este trabajo, mientras que las últimas capas convolucionales de VGG16 se han reajustado mediante fine-tuning.

Código 4.2: Definición, configuración y compilación del modelo completo (VGG16 con ajuste fino + cabeza clasificadora multietiqueta).

```
x = tf.keras.layers.GlobalAveragePooling2D()(vgg.output)
x = tf.keras.layers.Dense(512, activation='relu')(x)
x = tf.keras.layers.Dropout(0.15)(x)
salida = tf.keras.layers.Dense(len(PATRONES), activation='sigmoid')(x)

modelo_completo = tf.keras.models.Model(inputs=vgg.input, outputs=salida)
```

Sus capas, en orden, son:

1. **Input**, la propia imagen ya preprocesada, de forma (256, 256, 3). Esta entrada atraviesa primero las capas convolucionales de VGG16, transformándose en un tensor de características de forma (8, 8, 512) antes de llegar a las capas de esta cabeza.
2. **GlobalAveragePooling2D**. Esta capa reduce el tensor (8, 8, 512) que sale de VGG16 y calcula la media de cada uno de los 512 canales, quedándose con un vector de 512 valores. Así hay muchos menos parámetros en las capas siguientes, y la predicción ya no depende de en qué píxel se ha activado un filtro, sino de su activación media en toda la imagen. Colocarla justo a la salida de VGG16 permite transformar la representación espacial en un vector compacto antes de la clasificación.
3. **Dense de 512 unidades, activación ReLU**. Al colocarse después del `GlobalAveragePooling2D`, esta capa opera sobre el vector de 512 valores ya reducido, combinando la información de todos los canales entre sí para obtener una representación de nivel más alto antes de la clasificación final.
4. **Dropout de tasa 0.15**. Durante el entrenamiento, desactiva aleatoriamente el 15 % de las 512 salidas de la capa anterior en cada pasada, forzando a la red a no depender en exceso de un pequeño subconjunto de neuronas y reduciendo así el riesgo de sobreajuste, especialmente relevante dado el tamaño reducido del conjunto de datos disponible.
5. **Dense de 7 unidades (una por cada patrón dermatoscópico), activación sigmoide**. A diferencia de una activación *softmax*, que fuerza a que las salidas sumen 1 y por tanto solo tiene sentido cuando las clases son mutuamente excluyentes, la activación sigmoide produce una probabilidad independiente

entre 0 y 1 para cada uno de los siete patrones. Esto es imprescindible en este trabajo porque los patrones dermatoscópicos no son mutuamente excluyentes: una misma imagen puede presentar varios patrones a la vez, por lo que la tarea es de clasificación multietiqueta y no de clasificación multiclase.

4.5. Función de pérdida y optimizador

Al tratarse de una clasificación multietiqueta con 7 salidas sigmoides independientes, la función de pérdida empleada es la entropía cruzada binaria (`binary_crossentropy`), aplicada de forma independiente a cada una de las 7 salidas y promediada entre ellas, en lugar de la entropía cruzada categórica que se usaría si las clases fueran excluyentes. Como métrica de seguimiento durante el entrenamiento se emplea `binary_accuracy`, que también se calcula de forma independiente para cada una de las 7 salidas.

Código 4.3: Configuración del optimizador Adam y compilación del modelo completo.

```
# Optimizador Adam con correccion de parametros estandar
optimizador = tf.keras.optimizers.Adam(learning_rate=0.00001, beta_1=0.9, beta_2
    =0.999, epsilon=1e-8, amsgrad=True)

modelo_completo.compile(loss='binary_crossentropy', optimizer=optimizador, metrics
    =['binary_accuracy'])
modelo_completo.summary()
```

Como optimizador se utiliza Adam, con los siguientes hiperparámetros [27]:

- **Tasa de aprendizaje (*learning rate*) de 0.00001**, muy inferior al valor por defecto de Adam (0.001) que sí se emplearía si VGG16 permaneciera completamente congelada. Como las últimas capas convolucionales de VGG16 ahora también son entrenables, una tasa de aprendizaje alta destruiría en las primeras iteraciones los pesos preentrenados en ImageNet que precisamente se querían aprovechar; una tasa mucho más baja permite ajustarlos de forma gradual y estable.
- $\beta_1 = 0.9$ y $\beta_2 = 0.999$, también los valores estándar de Adam, que controlan respectivamente la inercia del promedio móvil del gradiente y de su varianza.
- $\epsilon = 10^{-8}$, una constante pequeña que se suma en el denominador de la regla de actualización de Adam únicamente para evitar una división por cero cuando la varianza estimada del gradiente es prácticamente nula.
- **`amsgrad=True`**, una variante de Adam que mantiene el máximo histórico de la varianza estimada del gradiente en lugar de su promedio móvil, lo que da lugar a una convergencia más estable y evitando que la tasa de aprendizaje efectiva crezca de forma indebida en las últimas fases del entrenamiento [28].

Nótese que el optimizador actualiza tanto los pesos de la cabeza de clasificación como los de las últimas 5 capas descongeladas de VGG16; el resto de la red, que concentra una parte importante de los parámetros permanece congelada, lo que sigue ayudando a mitigar el riesgo de sobreajuste con un conjunto de datos de tamaño reducido, pese a que ahora hay más pesos entrenables que en un esquema de extracción de características puro.

5 Mapas de activación (Grad-CAM)

Uno de los objetivos de este trabajo es que la clasificación de patrones dermatoscópicos vaya acompañada de una explicación visual: un mapa que señale en qué zona de la imagen se ha fijado la red para decidir que un determinado patrón está presente.

Este capítulo describe la metodología empleada para dotar de interpretabilidad visual al clasificador adaptado VGG16. Para ello, se implementa el algoritmo Grad-CAM (*Gradient-weighted Class Activation Mapping*), una técnica de Inteligencia Artificial Explicable (XAI) que permite localizar visualmente en el espacio las regiones de la imagen dermatoscópica que justifican la activación de un determinado patrón clínico, en nuestro caso, de los patrones relacionados con el carcinoma basocelular.

5.1. Fundamento teórico de Grad-CAM

La generación de los mapas de calor de este proyecto se fundamenta en el algoritmo Grad-CAM (*Gradient-weighted Class Activation Mapping*), propuesto originalmente por Selvaraju *et al.* [29]. A diferencia de técnicas de explicabilidad previas que obligaban a alterar la arquitectura estructural, Grad-CAM permite obtener explicaciones visuales de cualquier modelo convolucional sin necesidad de reentrenamiento.

La técnica Grad-CAM aprovecha el flujo de gradientes que circula por la red durante la etapa de retropropagación (*backpropagation*) para calcular la importancia de cada neurona en las capas convolucionales finales. Las primeras capas de VGG16 capturan características de baja frecuencia como bordes, texturas básicas o vello, y permanecen congeladas con los pesos de ImageNet a lo largo de todo el entrenamiento. Sin embargo, es en las últimas capas convolucionales de la red, las mismas que se descongelan y reentrenan mediante *fine-tuning*, donde se retiene la información espacial más adaptada al dominio dermatoscópico, justo antes de que la capa `GlobalAveragePooling2D` reduzca ese mapa de activación en el vector que recibe la cabeza clasificadora. En este trabajo, la capa sobre la que se calcula el mapa de Grad-CAM es configurable (`CAPA_GRAD_CAM`), pudiendo elegirse, entre otras, `block3_pool` (32×32), `block4_pool` (16×16) o `block5_pool` (8×8 , la capa por defecto): cuanto más profunda la capa, menor es la resolución espacial del mapa resultante, pero mayor su relación con la decisión final de la red. En este trabajo se utiliza por defecto `block5_pool` para Grad-CAM, ya que proporciona una representación profunda de las características aprendidas por las últimas capas convolucionales, aunque tenga una resolución espacial limitada.

Para generar el mapa de calor para una clase o patrón dermatoscópico concreto c , primero se calcula el gradiente de la puntuación de dicha clase (Y^c) con respecto a las activaciones del mapa de características A^k de la capa convolucional seleccionada. Estos gradientes se promedian espacialmente mediante una operación de *global average pooling* para obtener los pesos de importancia de los canales (α_k^c), tal como se define en la Ecuación 5.1:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial Y^c}{\partial A_{i,j}^k} \quad (5.1)$$

Donde Z representa el área del mapa de características (el producto de su ancho y su alto), e i, j son las coordenadas espaciales del mapa. Una vez calculados los pesos, se realiza una combinación lineal ponderada de todos los mapas de características y se aplica la función de activación de rectificación lineal (ReLU), siguiendo la formulación de la Ecuación 5.2:

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \quad (5.2)$$

La aplicación de la función ReLU es un paso crítico en ingeniería de imagen médica, ya que discrimina de forma automática las características que reducen la puntuación de la clase, quedándose sólo con los píxeles cuyas intensidades y gradientes influyen de manera positiva en la presencia del patrón dermatoscópico analizado. El mapa bidimensional resultante se normaliza y se superpone sobre la imagen original en RGB, utilizando un mapa de color, para una mejor interpretación visual por parte del dermatólogo.

5.2. Implementación del algoritmo Grad-CAM en Python

Para la implementación de Grad-CAM, se ha hecho uso de la API de TensorFlow. A diferencia de un esquema con VGG16 completamente congelada, donde haría falta combinar dos modelos independientes, uno para VGG16 y otro para la cabeza de clasificación, aquí `modelo_completo` ya integra ambas partes de principio a fin, puesto que sus últimas capas también se han reentrenado. Por ello basta con definir un único modelo intermedio que exponga, junto a la predicción final, las activaciones de la capa elegida en `CAPA_GRAD_CAM`.

En el Código 5.1 se detalla la función desarrollada para el cálculo y extracción de los mapas de calor.

Código 5.1: Definición del modelo intermedio de Grad-CAM e implementación del cálculo de gradientes espaciales y generación del mapa de activación.

```
# Capa sobre la que se calcula el mapa de Grad-CAM: 'block5_pool'
# (8x8, por defecto), 'block4_pool' (16x16) o 'block3_pool' (32x32).
CAPA_GRAD_CAM = 'block5_pool'

capa_conv_grad_cam = modelo_completo.get_layer(CAPA_GRAD_CAM)
modelo_grad_cam = tf.keras.models.Model(
    inputs=modelo_completo.input,
    outputs=[capa_conv_grad_cam.output, modelo_completo.output],
)

def hacer_grad_cam(imagen, indice_patron):
    imagen_prep = tf.keras.applications.vgg16.preprocess_input(imagen.astype('float32'))
    entrada = np.expand_dims(imagen_prep, axis=0)

    with tf.GradientTape() as cinta:
        mapas_conv, predicciones = modelo_grad_cam(entrada)
        salida_patron = predicciones[:, indice_patron]

    # Gradiente de la salida del patron respecto a los mapas de activacion de VGG16
    :
    # indica la variacion de la prediccion frente a perturbaciones en cada filtro.
    gradientes = cinta.gradient(salida_patron, mapas_conv)
```

```

# Importancia de cada uno de los filtros: calculo de la media espacial
peso_por_filtro = tf.reduce_mean(gradientes[0], axis=(0, 1)).numpy()

mapas_conv = mapas_conv[0].numpy()
alto_mapa, ancho_mapa, num_filtros = mapas_conv.shape

mapa_calor = np.zeros((alto_mapa, ancho_mapa))
for filtro in range(num_filtros):
    mapa_calor = mapa_calor + peso_por_filtro[filtro] * mapas_conv[:, :, filtro]

# Aplicacion de la funcion ReLU para aislar contribuciones positivas y
# normalizacion
mapa_calor = np.maximum(mapa_calor, 0)
if mapa_calor.max() > 0:
    mapa_calor = mapa_calor / mapa_calor.max()

return mapa_calor

```

La función realiza los siguientes pasos clave:

- **Modelo intermedio de Grad-CAM:** A partir de `modelo_completo` ya entrenado, se construye un nuevo modelo, `modelo_grad_cam`, indicando como salidas tanto las activaciones de `CAPA_GRAD_CAM` como la predicción final de los siete patrones, sin necesidad de duplicar ni volver a ensamblar la red.
- **Preprocesado y expansión del tensor:** La imagen de entrada se adapta a las exigencias de la red preentrenada mediante `preprocess_input`, y se expande su dimensión espacial (`expand_dims`) para simular un lote (*batch*) de tamaño 1.
- **Diferenciación automática (GradientTape):** Se emplea el contexto `GradientTape` de TensorFlow para registrar las operaciones del paso hacia adelante (*forward pass*). Esto es estrictamente necesario para poder calcular posteriormente las derivadas parciales de la predicción respecto a las activaciones convolucionales.
- **Cálculo de pesos de importancia:** Mediante la función `tf.reduce_mean`, se realiza el *Global Average Pooling* sobre los gradientes espaciales, obteniendo un vector unidimensional donde cada elemento representa la importancia relativa de un filtro específico de `CAPA_GRAD_CAM` para el patrón evaluado.
- **Ponderación de mapas de características:** Se inicializa un tensor nulo (`mapa_calor`) sobre el cual se acumulan linealmente los mapas de activación de dicha capa, escalados por su correspondiente peso de importancia.
- **Rectificación y normalización:** Siguiendo el algoritmo estándar, se aplica una operación análoga a la función de activación ReLU (`np.maximum`) para descartar las características que restan probabilidad a la clase. Finalmente, el mapa de calor se normaliza al intervalo $[0, 1]$ para permitir su posterior mapeo de color y superposición sobre la fotografía clínica original.

6 Resultados

En este capítulo se exponen y analizan los resultados obtenidos tras el entrenamiento y validación de la red neuronal VGG16 adaptada para la detección del carcinoma basocelular (BCC). En primer lugar, se describen los criterios matemáticos y las métricas estadísticas empleadas para evaluar el rendimiento del clasificador. Posteriormente, se presenta la matriz de confusión global y se discute el impacto del algoritmo frente a la clasificación clínica.

6.1. Metodología de evaluación

La red produce, para cada imagen, siete salidas independientes (una por cada patrón dermatoscópico considerado: *PigmentNetwork*, *Ulceration*, *Large_B_G_OvoidNests*, *Multi_B_G_Globules*, *MapleLeaflike*, *SpokeWheel* y *ArborizingTelangiectasia*), cada una con activación *sigmoid*. Como los siete patrones no son excluyentes entre sí, una misma lesión puede presentar varios simultáneamente, se aplica un umbral (por defecto 0.5) a cada una de las siete salidas de forma independiente, y se calculan verdaderos/falsos positivos y negativos (TP/FP/TN/FN) para cada patrón por separado, de los que se derivan la sensibilidad y la especificidad.

Para la comprobación conjunta BCC / No-BCC se sigue además el criterio clínico empleado por los dermatólogos: una lesión se clasifica como BCC si **no** presenta patrón reticular y presenta **al menos uno** de los otros seis patrones[23].

6.2. Conjunto de Datos Utilizado

Las imágenes empleadas para el entrenamiento, validación y evaluación de la red neuronal se organizan en varios directorios, definidos según su procedencia y finalidad dentro del flujo de trabajo:

- **Directorio `datos_aumentados` (5 792 imágenes):** Constituye la base principal del conjunto de entrenamiento. Cada imagen presenta una resolución de 256×256 píxeles y un encuadre ajustado sobre la región de interés que contiene el patrón (a diferencia de las imágenes de evaluación clínica, que muestran la lesión completa junto a la piel sana circundante). La nomenclatura de cada archivo codifica directamente los patrones presentes en la imagen. Por ejemplo, un archivo cuyo nombre contiene la subcadena `PigmentNetwork_SpokeWheel` indica la presencia simultánea de ambos criterios. Esto permite construir de forma automatizada un vector multietiqueta de siete posiciones (una por cada patrón) que la red aprenderá a predecir. Dado que los patrones no son mutuamente excluyentes, una misma imagen puede tener múltiples etiquetas activas. La Tabla 6.1 detalla la distribución de imágenes para cada uno de los siete patrones en este directorio.

- **Directorio `datos_test_reticulo` (27 imágenes):** Imágenes reales etiquetadas exclusivamente con el patrón reticular activo (`PigmentNetwork` = 1, resto de patrones a cero). Representan la exposición de la red durante el entrenamiento a fotografías reales, a las que hacemos un recorte centrado para captar mejor la lesión. Los resultados presentados en este capítulo corresponden al modelo entrenado con estas 27 imágenes originales.
- **Directorio `datos_test_sinPatron` (79 imágenes):** Imágenes reales sin ningún patrón característico del BCC (vector de etiquetas nulo). A las imágenes de este directorio también les hacemos un recorte centrado, ya que muchas muestran la lesión pequeña dentro de la imagen junto con bastante ruido de fondo, como el borde oscuro de la lente del dermatoscopio.
- **Directorios `BCC_ISIC` (50 imágenes) y `NoBCC_ISIC` (50 imágenes):** Componen el conjunto de evaluación externo y completamente independiente de la fase de entrenamiento, con diagnóstico clínico confirmado mediante el fichero de diagnóstico oficial del *ISIC 2018 Challenge*, concretamente de su Tarea 3 (*Task 3: Lesion Diagnosis*). Cruzando dicho fichero con las imágenes correspondientes del conjunto de entrenamiento de la Tarea 3 (10 015 imágenes en total, de las cuales 514 están etiquetadas como carcinoma basocelular), se seleccionaron 50 imágenes de BCC para el directorio `BCC_ISIC`. Para el directorio `NoBCC_ISIC` no se realizó un muestreo puramente aleatorio de las imágenes restantes, para que un único tipo de lesión benigna no domine el conjunto no-BCC; en su lugar, las 50 imágenes se repartieron entre los distintos tipos de lesiones no-BCC disponibles, para obtener un conjunto de control variado.

Tabla 6.1: Frecuencia de aparición de cada patrón en el directorio `datos_aumentados` (5 792 imágenes). Al no ser clases excluyentes, la suma de las frecuencias es superior al total de imágenes.

Patrón	N.º de imágenes
Retículo pigmentado (<i>PigmentNetwork</i>)	1 122
Ulceración (<i>Ulceration</i>)	2 443
Nidos ovoides grandes azul-gris (<i>Large_B_G_OvoidNests</i>)	1 799
Múltiples glóbulos azul-gris (<i>Multi_B_G_Globules</i>)	2 458
Áreas en hoja de arce (<i>MapleLeaflike</i>)	1 546
Estructuras en rueda de carro (<i>SpokeWheel</i>)	1 067
Telangiectasias ramificadas (<i>ArborizingTelangiectasia</i>)	2 934

Cabe destacar que las imágenes de entrenamiento presentan técnicas de aumento de datos (*data augmentation*). Debido a la limitación inherente en la disponibilidad de fotografías clínicas etiquetadas, el conjunto original fue ampliado mediante transformaciones sintéticas geométricas. Esta práctica es un estándar en imagen médica para mitigar el riesgo de sobreajuste (*overfitting*). El presente proyecto no ejecuta la generación sintética de estas imágenes, sino que toma este conjunto ampliado como punto de partida.

De la totalidad de imágenes destinadas al desarrollo del modelo (`datos_aumentados` junto con `datos_test_reticulo` y `datos_test_sinPatron`, sumando un total de 5 898 imágenes), se aplica una partición aleatoria tras barajar el orden del conjunto de datos: un 70 % se reserva para el entrenamiento (*train*), un 20 % para la validación (*validation*) y un 10 % para el test interno (*test*). Por el contrario, los directorios `BCC_ISIC` y `NoBCC_ISIC` se excluyen deliberadamente de esta partición, reservándose en su totalidad como conjuntos de test externos para evaluar de forma objetiva la capacidad de generalización del sistema ante imágenes de procedencia clínica ajena al entrenamiento.

6.3. Métricas estadísticas de evaluación

En el ámbito del diagnóstico médico asistido por ordenador, evaluar el desempeño de un algoritmo basándose exclusivamente en el porcentaje de aciertos globales puede resultar engañoso, especialmente si el

conjunto de datos de prueba se encuentra desbalanceado. Por ello, es imperativo recurrir a un conjunto de métricas estadísticas que permitan auditar de forma rigurosa la capacidad diagnóstica del modelo.

Estas métricas se calculan a partir de cuatro parámetros fundamentales:

- **Verdaderos Positivos (TP):** Imágenes en las que el patrón está presente y es correctamente detectado.
- **Verdaderos Negativos (TN):** Imágenes en las que el patrón está ausente y la red lo clasifica como ausente.
- **Falsos Positivos (FP):** Imágenes en las que el patrón está ausente, pero la red lo activa.
- **Falsos Negativos (FN):** Imágenes en las que el patrón está presente, pero la red no lo detecta.

A continuación, se detallan las formulaciones matemáticas que rigen el cálculo automático de estos indicadores durante la fase de test en el entorno de desarrollo.

6.3.1. Exactitud Global (*Accuracy*)

La exactitud representa el porcentaje total de valores correctamente clasificados sobre el total del conjunto de prueba. Su expresión matemática se define en la Ecuación 6.1:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6.1)$$

6.3.2. Sensibilidad (*Sensitivity*)

La sensibilidad mide la proporción de casos positivos reales (la presencia de un patrón dermatoscópico concreto, o de un carcinoma basocelular en la clasificación clínica global) que fueron identificados correctamente por la red neuronal convolucional. Se calcula de acuerdo a la Ecuación 6.2:

$$\text{Sensitivity} = \frac{TP}{TP + FN} \quad (6.2)$$

6.3.3. Especificidad (*Specificity*)

La especificidad define la tasa de verdaderos negativos, es decir, la capacidad del algoritmo para reconocer con éxito los casos negativos (la ausencia de un patrón dermatoscópico concreto, o una lesión No-BCC en la clasificación clínica global). Su formula matemática se detalla en la Ecuación 6.3:

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (6.3)$$

6.3.4. Valor Predictivo Positivo (*Precision*)

El Valor Predictivo Positivo (*Precision*) Expresa la proporción de predicciones de clase positiva correctas, sobre el total de predicciones de clase positiva, tanto correctas como incorrectas. (Ecuación 6.4).

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6.4)$$

6.3.5. F1-score

La métrica *F1-score* resume el compromiso entre sensibilidad y precisión en un único valor, calculando la media armónica entre ambas (Ecuación 6.5). A diferencia de la media aritmética, la media armónica penaliza fuertemente que una de las dos métricas sea baja, aunque la otra sea muy alta, por lo que solo alcanza valores elevados cuando sensibilidad y precisión son buenas simultáneamente.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}} = \frac{2TP}{2TP + FP + FN} \quad (6.5)$$

6.3.6. Matriz de confusión

La matriz de confusión es una representación visual, en forma de tabla, que compara los valores reales frente a los valores predichos por el modelo para cada una de las clases del problema; constituye la base sobre la que se construyen el resto de métricas descritas en esta sección. Enfrenta visualmente la clase predicha por el modelo con la clase real de cada imagen, permitiendo identificar de un vistazo si los errores del sistema se concentran en los falsos positivos o en los falsos negativos.

6.4. Evolución del entrenamiento

Durante el ajuste fino de las últimas 5 capas de VGG16, se entrenó el modelo final durante 20 épocas, monitorizando en cada una de ellas el `binary_accuracy` y la función de pérdida (`binary_crossentropy`) tanto sobre el conjunto de entrenamiento como sobre el de validación. Al tratarse de una salida de 7 unidades sigmoideas independientes, ambas métricas se calculan de forma independiente para cada patrón y se promedian entre las 7 salidas, por lo que los valores mostrados representan la media conjunta de los siete patrones y no el resultado de un único clasificador binario. La Figura 6.1 muestra la evolución de ambas métricas promediadas a lo largo del entrenamiento.

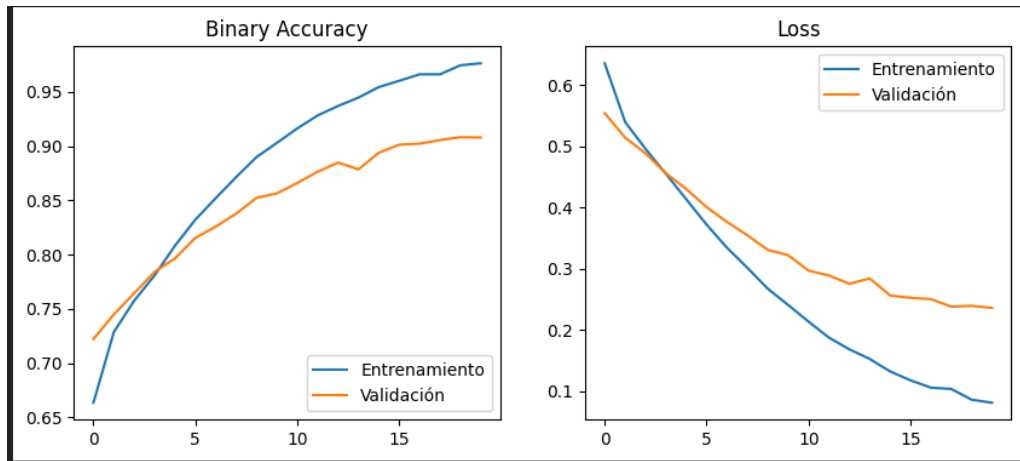


Figura 6.1: Evolución del *binary accuracy* y de la función de pérdida durante el entrenamiento, sobre el conjunto de entrenamiento y el de validación.

Durante las primeras épocas, tanto el *accuracy* como la pérdida mejoran de forma prácticamente pareja en entrenamiento y validación (en la época 5, por ejemplo, el *accuracy* de entrenamiento es 0.808 frente a 0.797 en validación). A partir de la época 10 aproximadamente, esta diferencia empieza a ensancharse: el *accuracy* de entrenamiento sigue subiendo de forma casi lineal hasta 0.976 en la época 20, mientras que el

de validación se va estancando en torno a 0.91 desde la época 16 en adelante (0.901, 0.902, 0.906, 0.9082 y 0.9080 en las últimas cinco épocas). La pérdida sigue la misma tendencia: la de validación apenas mejora en el mismo tramo final (de 0.252 a 0.236), mientras que la de entrenamiento continúa descendiendo con claridad (de 0.118 a 0.081).

Este ensanchamiento progresivo entre las curvas de entrenamiento y de validación, el modelo sigue ajustándose cada vez mejor al conjunto de entrenamiento sin que ello se traduzca ya en una mejora comparable sobre datos no vistos, es la señal característica de sobreajuste (*overfitting*). Por ello se decidió detener el entrenamiento en la época 20: continuar entrenando habría seguido reduciendo la pérdida de entrenamiento sin aportar una mejora real de generalización, a costa de que el modelo memorizase cada vez más particularidades del conjunto de entrenamiento poco útiles, o incluso perjudiciales, fuera de él.

6.5. Resultados de clasificación por patrón (Test interno)

La primera fase de evaluación se realizó sobre un conjunto de test interno (10 % de las imágenes reservadas del conjunto de entrenamiento). La Tabla 6.2 muestra la sensibilidad y especificidad para cada patrón con el umbral por defecto de 0.5, tras el ajuste fino de las últimas 5 capas de VGG16 durante 20 épocas.

Tabla 6.2: Sensibilidad y especificidad por patrón dermatoscópico (Test interno).

Patrón	TP	FP	TN	FN	Sensibilidad	Especificidad
PigmentNetwork	73	1	475	40	0.65	1.00
Ulceration	217	47	313	12	0.95	0.87
Large_B_G_OvoidNests	115	13	406	55	0.68	0.97
Multi_B_G_Globules	194	37	326	32	0.86	0.90
MapleLeaflike	113	10	420	46	0.71	0.98
SpokeWheel	83	4	479	23	0.78	0.99
ArborizingTelangiectasia	264	20	253	52	0.84	0.93

Los resultados muestran un perfil muy distinto al de versiones anteriores del modelo: `PigmentNetwork` alcanza una especificidad prácticamente perfecta (1.00), señal de que apenas se activa de forma indebida, mientras que `Ulceration` y `Multi_B_G_Globules` obtienen las sensibilidades más altas (0.95 y 0.86 respectivamente).

La Tabla 6.2 puede representarse también en forma de matriz de confusión para cada uno de los siete patrones (Figura 6.2), lo que permite visualizar de un vistazo si los errores de cada patrón se concentran en los falsos positivos o en los falsos negativos.

6.6. Clasificación clínica global (BCC frente a No-BCC)

En el análisis individual de los patrones, los conceptos de TP, FP, TN y FN se refieren a la presencia o ausencia del patrón dermatoscópico evaluado. En cambio, para la clasificación clínica global, estos mismos conceptos se aplican a la decisión BCC/No-BCC.

Para evaluar la capacidad de generalización del sistema, se utilizó el conjunto externo `BCC_ISIC/NoBCC_ISIC` (100 imágenes, 50 BCC y 50 No-BCC). Se ha aplicado el modelo final VGG16, con las últimas 5 capas de la base VGG16 descongeladas, entrenado durante 20 épocas, y se obtuvo la matriz de resultados reflejada en la Tabla 6.3.

A diferencia de versiones anteriores con las que se probó el modelo, donde se observaba una clara tendencia a sobre-predecir la presencia de BCC (sensibilidad alta a costa de una especificidad muy pobre), esta

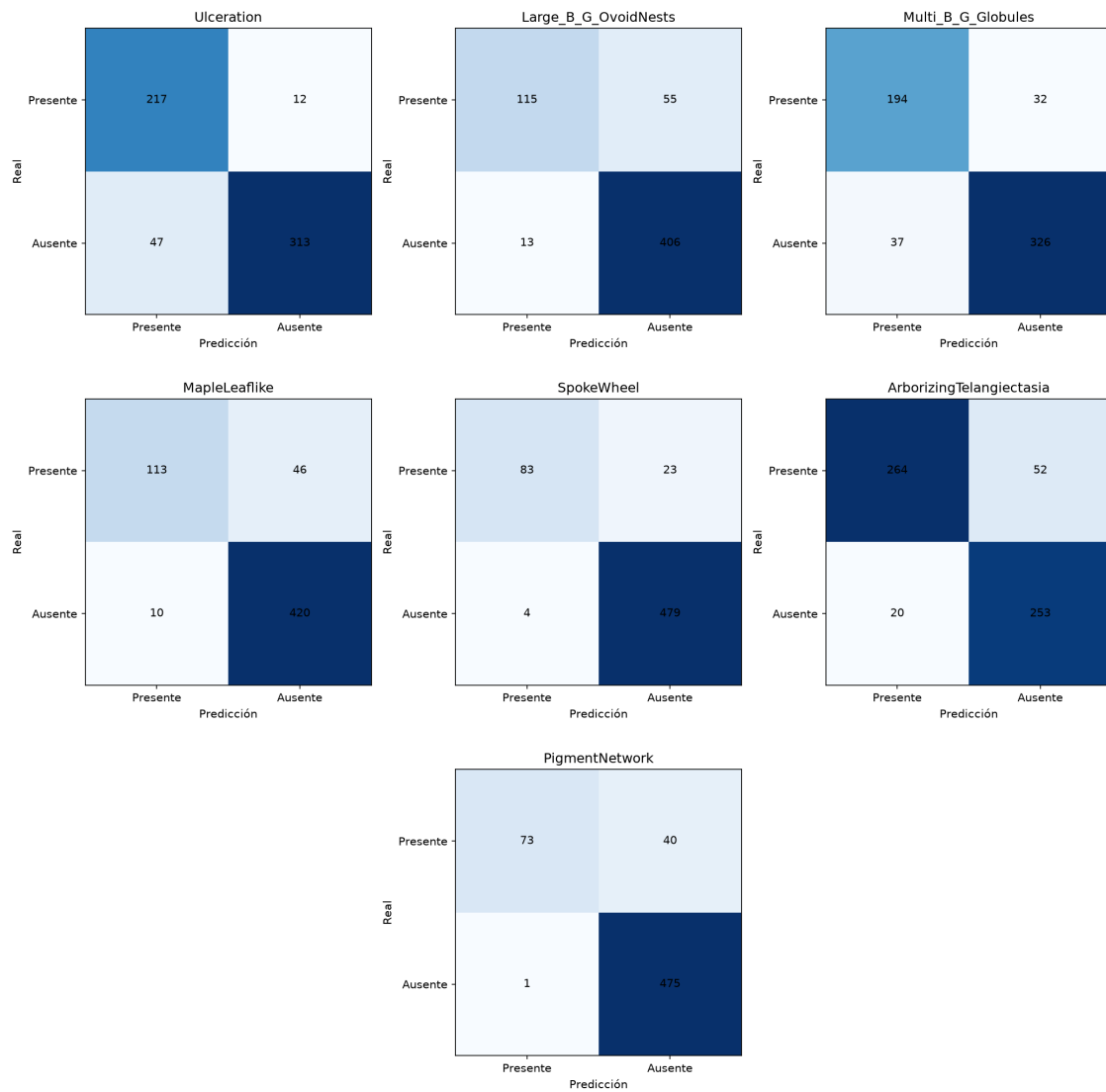


Figura 6.2: Matrices de confusión individuales para cada patrón dermatoscópico (Test interno).

Tabla 6.3: Métricas de clasificación global BCC / No-BCC.

Métrica	Valor
Verdaderos Positivos (TP)	49
Falsos Positivos (FP)	5
Verdaderos Negativos (TN)	45
Falsos Negativos (FN)	1
Accuracy (Exactitud)	0.94
Sensibilidad	0.98
Especificidad	0.90
Valor Predictivo Positivo	0.91
F1-score	0.94

configuración logra un equilibrio mucho más sólido entre ambas métricas, con un único falso negativo sobre el conjunto de 50 imágenes BCC.

La Tabla 6.3 puede representarse también en forma de matriz de confusión (Figura 6.3).

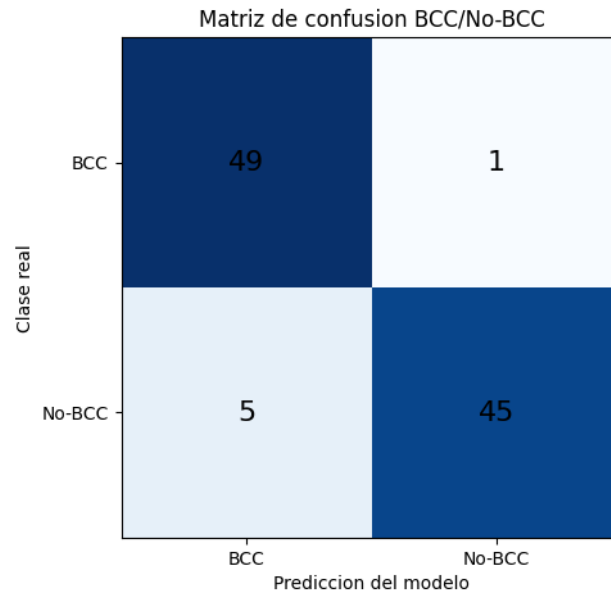


Figura 6.3: Matriz de confusión del modelo final sobre el conjunto de evaluación BCC/No-BCC.

6.7. Casos de estudio: Análisis visual con Grad-CAM sobre el conjunto BCC/No-BCC

Las métricas agregadas presentadas en la Tabla 6.3 cuantifican el rendimiento global del sistema, pero no proporcionan interpretabilidad sobre las causas subyacentes de los aciertos o errores en casos particulares. Para dotar al modelo de esta explicabilidad clínica, se han generado mapas de calor mediante Grad-CAM sobre imágenes que permitan representar distintas casuísticas relevantes.

El objetivo de este análisis es verificar visualmente si los patrones que activan el criterio de clasificación se fundamentan en estructuras dermatoscópicas reales de la lesión o si, por el contrario, la red sufre sesgos al apoyarse en regiones espaciales sin relevancia médica.

6.7.1. Carcinomas basocelulares con patrón correctamente identificado

En los siguientes ejemplos, todos ellos correspondientes a imágenes con diagnóstico real de carcinoma basocelular, el patrón dermatoscópico predominante de cada lesión supera el umbral de 0.5 y el mapa de calor Grad-CAM concentra su activación sobre la estructura clínica correspondiente en la imagen. Esta coincidencia espacial entre la zona de máxima activación y la evidencia dermatoscópica real sugiere que la red está utilizando información localizada en una región clínicamente relevante para identificar el patrón, y no en artefactos o regiones ajenas a la lesión.

Ulceration (ISIC_0027858)

La ulceración se manifiesta como una pérdida de continuidad de la epidermis; en este ejemplo, el mapa de calor se concentra precisamente sobre dicha zona erosionada, coincidiendo con el área clínicamente relevante para este patrón.

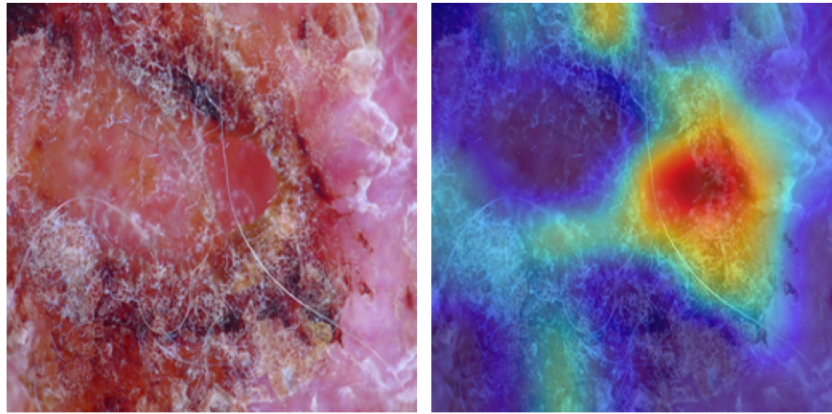


Figura 6.4: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0027858), con activación del patrón Ulceration correctamente localizada.

Large_B_G_OvoidNests (ISIC_0026321)

Los nidos ovoides grandes azul-gris aparecen como agrupaciones bien delimitadas de pigmento; el Grad-CAM sitúa su activación máxima sobre estas estructuras, confirmando que la red las reconoce como tales y no como ruido de fondo.

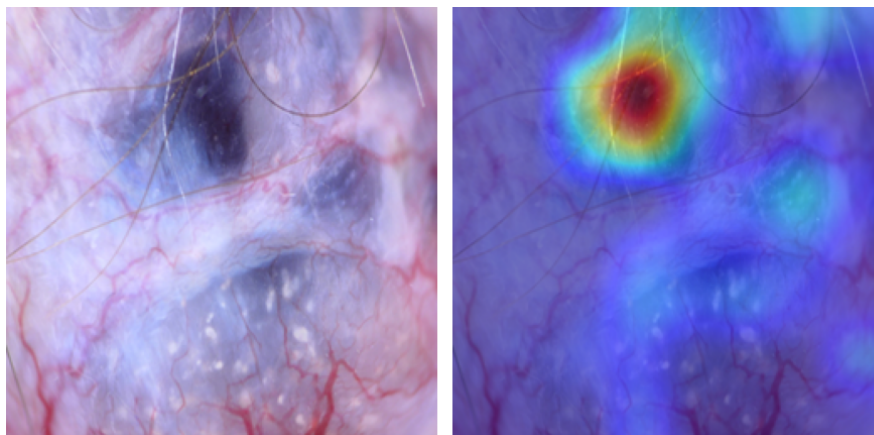


Figura 6.5: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0026321), con activación del patrón Large_B_G_OvoidNests correctamente localizada.

ArborizingTelangiectasia (ISIC_0028486)

Las telangiectasias arboriformes son vasos sanguíneos ramificados característicos del BCC; la activación del mapa de calor recae sobre el trazado de estos vasos, en lugar de dispersarse sobre el resto de la piel visible.

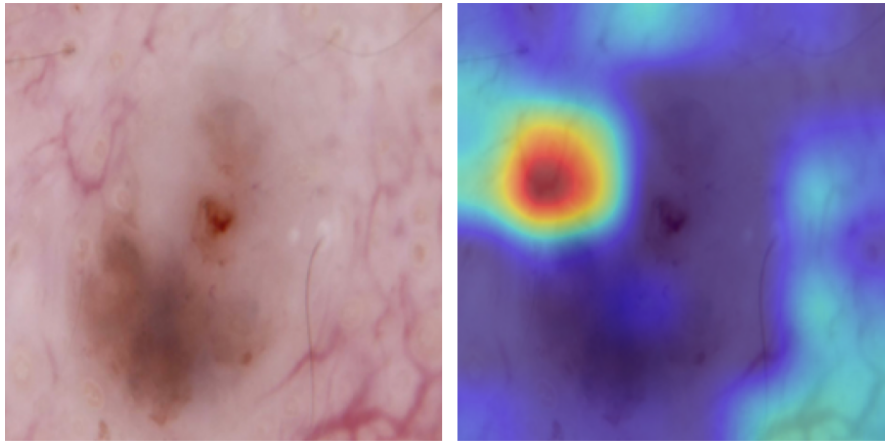


Figura 6.6: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0028486), con activación del patrón ArborizingTelangiectasia correctamente localizada.

MapleLeaflike (ISIC_0032414)

Las áreas en hoja de arce presentan un contorno pigmentado lobulado y bien definido; el mapa de calor coincide con la silueta de dicha estructura, mostrando que la red localiza su forma característica.

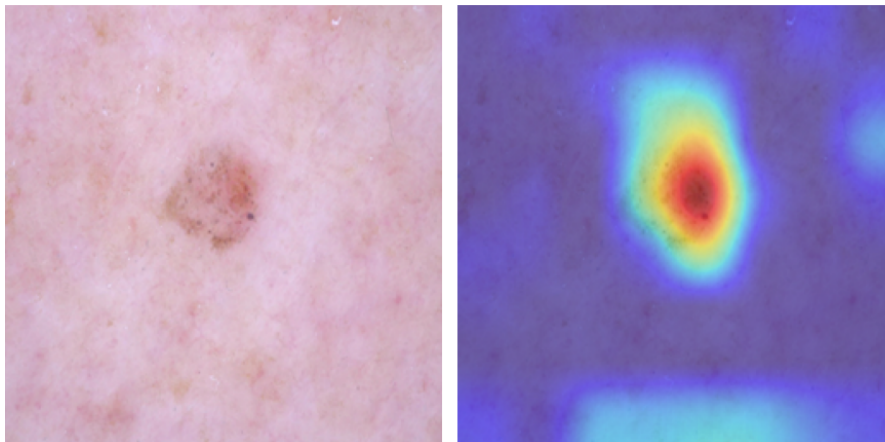


Figura 6.7: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0032414), con activación del patrón MapleLeaflike correctamente localizada.

Multi_B_G_Globules (ISIC_0034095)

Los múltiples glóbulos azul-gris se distribuyen de forma difusa por la lesión; en este caso, la activación del Grad-CAM también se reparte sobre las distintas agrupaciones de glóbulos presentes, en lugar de concentrarse en un único punto.

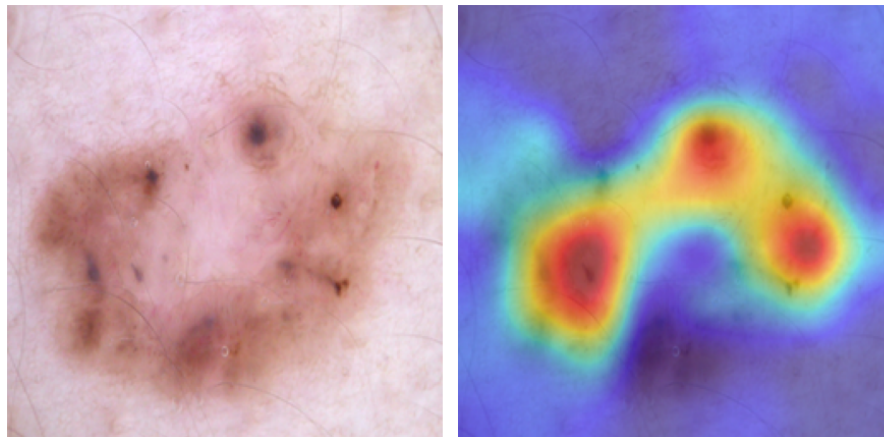


Figura 6.8: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0034095), con activación del patrón Multi_B_G_Globules correctamente localizada.

6.7.2. Carcinoma basocelular con activación errónea del patrón reticular

PigmentNetwork (ISIC_0026798)

El criterio clínico implementado exige la ausencia del patrón reticular (*PigmentNetwork*) para clasificar una lesión como BCC. Este carcinoma basocelular real, sin embargo, activa dicho patrón por encima del umbral. El mapa de calor revela que la activación se concentra sobre el vello corporal presente en la imagen: su disposición de líneas finas y entrecruzadas guarda cierto parecido visual con la retícula pigmentada característica de los nevos melanocíticos, lo que induce a la red a confundir ambas estructuras. Este error resulta especialmente relevante desde el punto de vista clínico, ya que si el resto de patrones de BCC no alcanzasen también el umbral, esta confusión bastaría por sí sola para producir un falso negativo, al activarse el único patrón que el criterio exige que esté ausente.

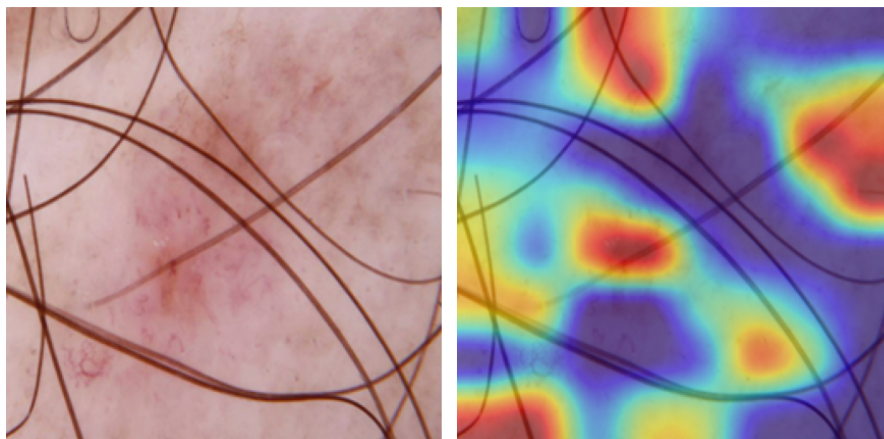


Figura 6.9: Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0026798), con activación del patrón PigmentNetwork erróneamente localizada sobre el vello corporal de la imagen.

6.7.3. Lesiones No-BCC con estructuras visualmente similares a patrones de BCC

Los siguientes ejemplos corresponden a lesiones sin diagnóstico de carcinoma basocelular sobre las que la red neuronal activa alguno de los seis patrones asociados al BCC. En estos casos el mapa de calor sitúa

el foco de atención de forma correcta, sobre estructuras reales de la lesión visualmente parecidas al patrón detectado; no se trata, por tanto, de un fallo de localización de la red, sino de una dificultad asociada al problema: existen lesiones No-BCC que presentan estructuras visualmente similares a las asociadas al BCC.

Ulceration (ISIC_0030119)

Esta lesión (identificador ISIC_0030119) presenta una zona que la red interpreta como ulceración, correctamente localizada sobre la erosión real de la lesión.

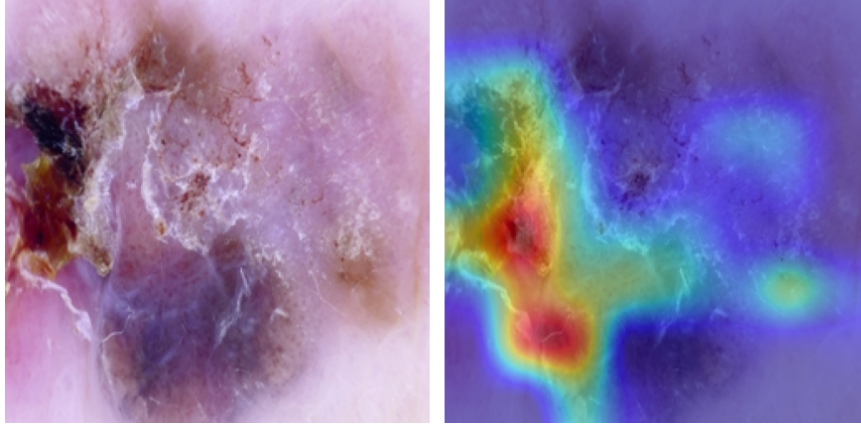


Figura 6.10: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0030119), con activación del patrón Ulceration correctamente localizada.

Large_B_G_OvoidNests (ISIC_0033762)

La lesión (ISIC_0033762) presenta una agrupación de pigmento que la red interpreta como nidos ovoides grandes azul-gris, de nuevo correctamente localizada por el Grad-CAM pese a tratarse de una lesión no-BCC.

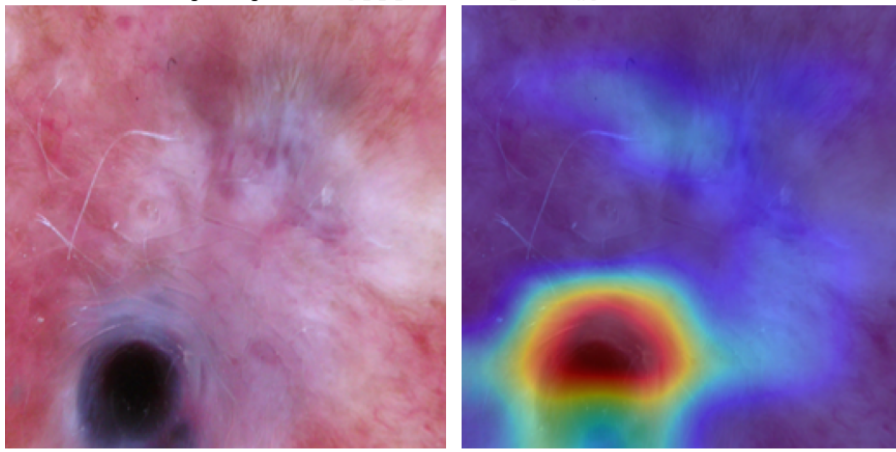


Figura 6.11: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0033762), con activación del patrón Large_B_G_OvoidNests correctamente localizada.

ArborizingTelangiectasia (ISIC_0029502)

En este otro caso, la lesión presenta una especie de artefacto con un borde fino similar al vello que la red confunde con telangiectasias arboriformes.

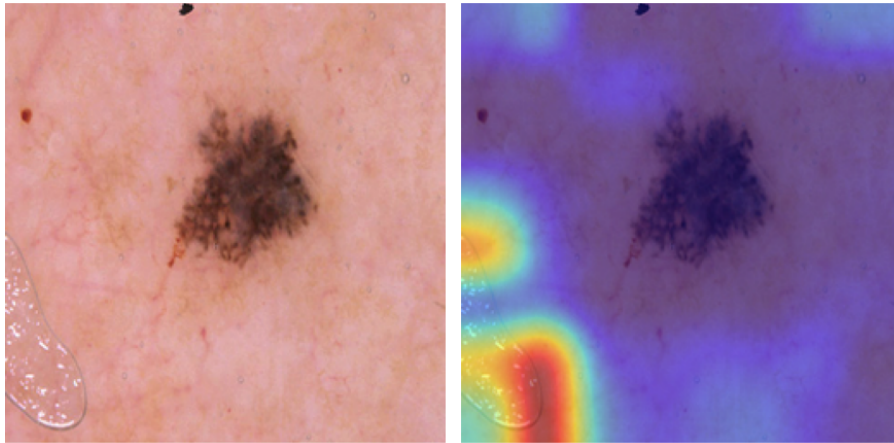


Figura 6.12: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0029502), con activación del patrón ArborizingTelangiectasia correctamente localizada.

MapleLeaflike (ISIC_0025911)

En esta imagen, aunque no se aprecia claramente en la imagen real una estructura de hojas de arce, si que los bordes de la lesión rosácea en tono marrón claro pueden confundirse con una estructura de hojas de arce.

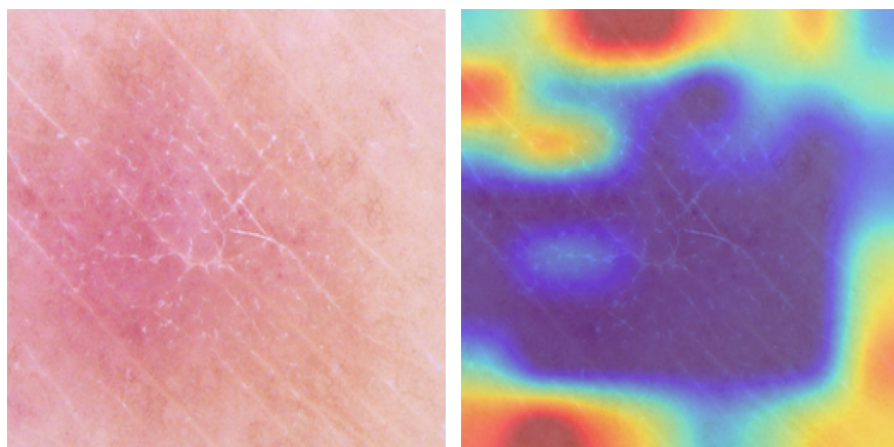


Figura 6.13: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0025911), con activación del patrón MapleLeaflike correctamente localizada.

Multi_B_G_Globules (ISIC_0024987)

Por último, esta lesión presenta vello fino junto, lo que la red identifica como múltiples glóbulos azul-gris.

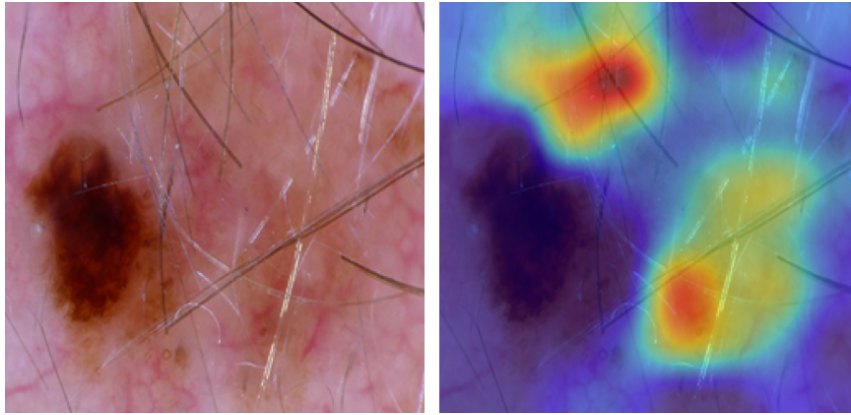


Figura 6.14: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0024987), con activación del patrón Multi_B_G_Globules correctamente localizada.

6.7.4. Lesión No-BCC correctamente descartada mediante el patrón reticular

PigmentNetwork (ISIC_0032087)

Por último, se muestra el caso de una lesión sin diagnóstico de BCC en la que el único patrón activado es el reticular (*PigmentNetwork*), correctamente localizado por el Grad-CAM sobre la retícula pigmentada real de la lesión. Al no activarse ningún otro patrón, el criterio clínico descarta directamente el BCC, ejemplificando el funcionamiento esperado de este patrón como criterio de exclusión.

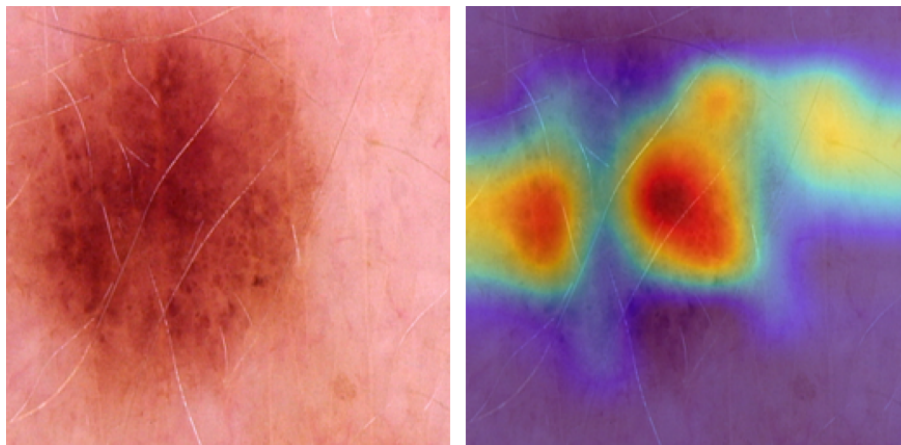


Figura 6.15: Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0032087), con activación del patrón PigmentNetwork correctamente localizada.

7 Conclusiones y líneas futuras

En este Trabajo Fin de Grado se ha desarrollado y evaluado un sistema de apoyo al diagnóstico del carcinoma basocelular (BCC) a partir de imágenes dermatoscópicas, basado en el ajuste fino de las últimas cinco capas de VGG16 sobre un problema de clasificación multietiqueta de siete patrones dermatoscópicos. Frente a los enfoques iniciales de red sin congelar, y red totalmente congelada, reentrenar parcialmente VGG16, ha permitido que sus filtros más específicos se adapten al dominio dermatoscópico, mejorando de forma notable los resultados obtenidos en las primeras versiones probadas a lo largo del proyecto.

Sobre el conjunto de evaluación externo `BCC_ISIC/NoBCC_ISIC` (Tabla 6.3), el criterio clínico implementado alcanza una exactitud de 0.94, una sensibilidad de 0.98 y una especificidad de 0.90. Al tratarse de imágenes obtenidas de forma independiente al entrenamiento, con un simple recorte centrado en la evaluación y sin el encuadre cercano sobre el patrón que sí tienen las imágenes de entrenamiento, estos resultados son un indicio razonable de que el sistema generaliza más allá del conjunto aumentado sobre el que se entrenó.

El análisis visual mediante Grad-CAM respalda esta conclusión: en la mayoría de los casos estudiados, la activación de cada patrón coincide con la estructura dermatoscópica real que debería justificarlo, lo que sugiere que, en los casos analizados, el modelo está utilizando regiones visualmente coherentes con los patrones identificados. Este mismo análisis también ha servido para detectar limitaciones concretas, como la confusión ocasional del patrón reticular con el vello corporal o la dificultad para distinguir estructuras muy similares entre lesiones de distinta naturaleza clínica; resultados que las métricas por sí solas no habrían sacado a la luz, y que confirman el valor de incorporar explicabilidad en un sistema pensado para un entorno médico.

El propio desarrollo del proyecto ha dejado claras varias vías de mejora. El conjunto de imágenes para los patrones reticular y sin patrón sigue siendo pequeño en comparación con el resto, por lo que ampliarlo con nuevas imágenes clínicas sería el paso más directo para mejorar la generalización, sobre todo en el patrón reticular, el de menor sensibilidad de todos (Tabla 6.2). El desequilibrio entre patrones en `datos_aumentados` también podría abordarse con técnicas de ponderación de clases en la función de pérdida, en lugar de depender solo del aumento de datos ya aplicado.

Este TFG parte directamente del artículo de Serrano et al. [2], que ya empleaba VGG16 para este mismo problema, combinando la imagen original con información de color cuantizado, aunque sin incluir ningún mapa de activación que visualizara en qué se apoya la red para clasificar. Una línea futura natural sería aplicar esta misma auditoría visual sobre su arquitectura, más compleja y con mejores métricas, para comprobar si sus aciertos están igualmente bien fundamentados espacialmente. Por último, toda la evaluación de este TFG se ha realizado sobre imágenes ya diagnosticadas previamente, por lo que probar el sistema junto a un dermatólogo en un caso de uso real sería un buen paso a seguir antes de plantear cualquier aplicación clínica.

Índice de Figuras

Figura 3.1	Ejemplos visuales de los 6 patrones dermatoscópicos característicos del carcinoma basocelular y del patrón negativo retículo pigmentado.	9
Figura 4.1	Arquitectura VGG16 [25].	12
Figura 4.2	Diagrama de bloques de la arquitectura VGG16.	13
Figura 6.1	Evolución del <i>binary accuracy</i> y de la función de pérdida durante el entrenamiento, sobre el conjunto de entrenamiento y el de validación.	24
Figura 6.2	Matrices de confusión individuales para cada patrón dermatoscópico (Test interno).	26
Figura 6.3	Matriz de confusión del modelo final sobre el conjunto de evaluación BCC/No-BCC.	27
Figura 6.4	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0027858), con activación del patrón Ulceration correctamente localizada.	28
Figura 6.5	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0026321), con activación del patrón Large_B_G_OvoidNests correctamente localizada.	28
Figura 6.6	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0028486), con activación del patrón ArborizingTelangiectasia correctamente localizada.	29
Figura 6.7	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0032414), con activación del patrón MapleLeaflike correctamente localizada.	29
Figura 6.8	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0034095), con activación del patrón Multi_B_G_Globules correctamente localizada.	30
Figura 6.9	Mapa de calor Grad-CAM sobre un carcinoma basocelular real (ISIC_0026798), con activación del patrón PigmentNetwork erróneamente localizada sobre el vello corporal de la imagen.	30
Figura 6.10	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0030119), con activación del patrón Ulceration correctamente localizada.	31
Figura 6.11	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0033762), con activación del patrón Large_B_G_OvoidNests correctamente localizada.	31
Figura 6.12	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0029502), con activación del patrón ArborizingTelangiectasia correctamente localizada.	32
Figura 6.13	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0025911), con activación del patrón MapleLeaflike correctamente localizada.	32
Figura 6.14	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0024987), con activación del patrón Multi_B_G_Globules correctamente localizada.	33
Figura 6.15	Mapa de calor Grad-CAM sobre una lesión sin diagnóstico de BCC (ISIC_0032087), con activación del patrón PigmentNetwork correctamente localizada.	33

Índice de Tablas

Tabla 4.1 Estructura de capas, dimensiones de salida y parámetros del modelo.	14
Tabla 6.1 Frecuencia de aparición de cada patrón en el directorio <code>datos_aumentados</code> (5 792 imágenes). Al no ser clases excluyentes, la suma de las frecuencias es superior al total de imágenes.	22
Tabla 6.2 Sensibilidad y especificidad por patrón dermatoscópico (Test interno).	25
Tabla 6.3 Métricas de clasificación global BCC / No-BCC.	26

Índice de Códigos

Código 4.1	Configuración de las épocas de entrenamiento y ajuste del modelo completo.	14
Código 4.2	Definición, configuración y compilación del modelo completo (VGG16 con ajuste fino + cabeza clasificadora multietiqueta).	15
Código 4.3	Configuración del optimizador Adam y compilación del modelo completo.	16
Código 5.1	Definición del modelo intermedio de Grad-CAM e implementación del cálculo de gradientes espaciales y generación del mapa de activación.	18
Código A.1	Importaciones, detección del entorno de ejecución y rutas de datos.	47
Código A.2	Rutas de las carpetas de datos, configuración global del experimento y función de carga y preprocesado (recorte y redimensionado) de una imagen.	47
Código A.3	Construcción del conjunto de imágenes de entrenamiento y reparto en entrenamiento, validación y test.	49
Código A.4	Carga en memoria de las imágenes de entrenamiento, validación y test.	50
Código A.5	Construcción del modelo VGG16 con ajuste fino de las últimas capas, cabeza clasificadora y compilación.	50
Código A.6	Preprocesado final de las imágenes y entrenamiento del modelo completo.	51
Código A.7	Curvas de entrenamiento: evolución del <i>binary accuracy</i> y de la función de pérdida.	51
Código A.8	Carga del modelo entrenado y de los datos de test guardados, sin necesidad de repetir el entrenamiento.	52
Código A.9	Clasificación del test interno mediante umbral y cálculo de sensibilidad y especificidad por patrón.	52
Código A.10	Matrices de confusión individuales para cada patrón dermatoscópico (test interno).	53
Código A.11	Evaluación sobre el conjunto externo BCC_ISIC/NoBCC_ISIC.	53
Código A.12	Matriz de confusión 2x2 del criterio clínico BCC/No-BCC, con umbral 0.5.	55
Código A.13	Definición del modelo intermedio de Grad-CAM y de las funciones para calcular y mostrar el mapa de calor.	55
Código A.14	Recorrido de las imágenes BCC/No-BCC con Grad-CAM, mostrando únicamente las que activan un patrón concreto.	56

Bibliografía

- [1] Skin Cancer Foundation, “Basal cell carcinoma.” [Online]. Available: <https://www.skincancer.org/skin-cancer-information/basal-cell-carcinoma/>
- [2] C. Serrano, M. Lazo, A. Serrano, T. Toledo-Pastrana, R. Barros-Bornay, and B. Acha, “Clinically inspired skin lesion classification through the detection of dermoscopic criteria for basal cell carcinoma,” *Journal of Imaging*, vol. 8, p. 197, 2022.
- [3] N. Chuchu, J. Dinnes, Y. Takwoingi, R. N. Matin, S. E. Bayliss, C. Davenport, J. F. Moreau, O. Bassett, K. Godfrey, C. O’Sullivan, F. M. Walter, R. Motley, J. J. Deeks, and H. C. Williams, “Teledermatology for diagnosing skin cancer in adults,” *Cochrane Database of Systematic Reviews*, no. 12, 2018. [Online]. Available: <https://doi.org/10.1002/14651858.CD013193>
- [4] F. Lubinus Badillo, C. A. Rueda Hernández, B. Marconi Narváez, and Y. E. Arias Trillos, “Redes neuronales convolucionales: un modelo de deep learning en imágenes diagnósticas. revisión de tema,” *Revista Colombiana de Radiología*, vol. 32, no. 3, pp. 5591–5599, 2021.
- [5] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun, “Dermatologist-level classification of skin cancer with deep neural networks,” *Nature*, vol. 542, no. 7639, pp. 115–118, 2017.
- [6] S. Tammina, “Transfer learning using vgg-16 with deep convolutional neural network for classifying images,” *International Journal of Scientific and Research Publications*, vol. 9, no. 10, 2019.
- [7] K. Tyagi, S. Vats, and V. Vashisht, “Implementing inception v3, vgg-16 and vgg-19 architectures of cnn for medicinal plant leaves identification and disease detection,” *Journal of Electrical Systems*, vol. 20, no. 7s, pp. 2380–2388, 2024.
- [8] S. Thanzeem Mohamed Sheriff, J. Venkat Kumar, S. Vigneshwaran, A. Jones, and J. Anand, “Lung cancer detection using VGG NET 16 architecture,” *Journal of Physics: Conference Series*, vol. 2040, no. 1, p. 012001, oct 2021. [Online]. Available: <https://doi.org/10.1088/1742-6596/2040/1/012001>
- [9] C. Vimal and N. Shirivastava, “Face and face-mask detection system using vgg-16 architecture based on convolutional neural network,” *International Journal of Computer Applications*, vol. 183, no. 50, 2022.
- [10] A. Marka, J. B. Carter, E. Toto, and S. Hassanpour, “Automated detection of nonmelanoma skin cancer using digital images: a systematic review,” *BMC Medical Imaging*, vol. 19, no. 21, pp. 1–12, 2019.
- [11] H. Huang, P. Kharazmi, D. McLean, H. Lui, Z. Wang, and T. Lee, “Automatic detection of translucency using a deep learning method from patches of clinical basal cell carcinoma images,” in *Proceedings, APSIPA Annual Summit and Conference*, 2018, pp. 685–688.

- [12] B. Cheng, D. Erdos, R. J. Stanley, W. V. Stoecker, D. A. Calcara, and D. D. Gómez, “Automatic detection of basal cell carcinoma using telangiectasia analysis in dermoscopy skin lesion images,” *Skin Research and Technology*, vol. 17, no. 3, pp. 278–287, 2011.
- [13] P. Guvenc, R. W. LeAnder, S. Kefel, W. V. Stoecker, R. K. Rader, K. A. Hinton, S. M. Stricklin, H. S. Rabinovitz, M. Oliviero, and R. H. Moss, “Sector expansion and elliptical modeling of blue-gray ovoids for basal cell carcinoma discrimination in dermoscopy images,” *Skin Research and Technology*, vol. 19, no. 1, pp. e532–e536, 2013.
- [14] S. Kefel, S. P. Kefel, R. W. LeAnder, R. Kaur, R. Kasmi, N. K. Mishra, R. K. Rader, J. G. Cole, Z. T. Woolsey, and W. V. Stoecker, “Adaptable texture-based segmentation by variance and intensity for automatic detection of semitranslucent and pink blush areas in basal cell carcinoma,” *Skin Research and Technology*, vol. 22, pp. 412–422, 2016.
- [15] P. Kharazmi, S. Kalia, H. Lui, Z. J. Wang, and T. K. Lee, “A feature fusion system for basal cell carcinoma detection through data-driven feature learning and patient profile,” *Skin Research and Technology*, vol. 24, pp. 256–264, 2018.
- [16] B. H. van der Velden, H. J. Kuijf, K. G. Gilhuijs, and M. A. Viergever, “Explainable artificial intelligence (xai) in deep learning-based medical image analysis,” *Medical Image Analysis*, vol. 79, p. 102470, 2022.
- [17] I. Ben Ahmed and C. Ben Amar, “Xai-cad: towards trustworthy explainable cad for breast ultrasound diagnosis,” *Journal of Information and Telecommunication*, 2026.
- [18] S. Iftikhar, N. Anjum, A. B. Siddiqui, M. U. Rehman, and N. Ramzan, “Explainable cnn for brain tumor detection and classification through xai based key features identification,” *Brain Informatics*, vol. 12, p. 10, 2025.
- [19] M. L. F. Jumaili and E. Sonuç, “An attention-based cnn framework for alzheimer’s disease staging with multi-technique xai visualization,” *Computers, Materials & Continua*, vol. 83, no. 2, 2025.
- [20] H. C. Yoon and L. P. Lin, “Brain tumor classification in mri: Insights from lime and grad-cam explainable ai techniques,” *IEEE Access*, vol. 13, pp. 154 172–154 202, 2025.
- [21] P. Tschandl, C. Rinner, Z. Apalla, G. Argenziano, N. Codella, A. Halpern, M. Janda, A. Lallas, C. Longo, J. Malvehy, J. Paoli, S. Puig, C. Rosendahl, H. P. Soyer, I. Zalaudek, and H. Kittler, “Human-computer collaboration for skin cancer recognition,” *Nature Medicine*, vol. 26, pp. 1229–1234, 2020.
- [22] P. Zaballos Diego, “Criterios dermatoscópicos de los tumores no melanocíticos,” *Medicina cutánea ibero-latino-americana*, vol. 43, no. 2, pp. 93–103, 2015.
- [23] S. W. Menzies, K. Westerhoff, H. Rabinovitz, A. W. Kopf, W. H. McCarthy, and B. Katz, “Surface microscopy of pigmented basal cell carcinoma,” *Archives of Dermatology*, vol. 136, no. 8, pp. 1012–1016, 2000.
- [24] K. Peris, E. Altobelli, A. Ferrari, M. C. Fagnoli, D. Piccolo, M. Esposito, and S. Chimenti, “Interobserver agreement on dermoscopic features of pigmented basal cell carcinoma,” *Dermatologic Surgery*, vol. 28, no. 7, pp. 643–645, 2002.
- [25] N. N. Prakash, V. Rajesh, D. L. Namakhwa, S. D. Pande, and S. H. Ahammad, “A densenet cnn-based liver lesion prediction and classification for future medical diagnosis,” *Scientific African*, vol. 20, p. e01629, 2023.
- [26] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [27] D. P. Kingma and J. L. Ba, “Adam: A method for stochastic optimization,” in *Published as a conference paper at ICLR 2015*, 2015.

- [28] S. J. Reddi, S. Kale, and S. Kumar, “On the convergence of adam and beyond,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [29] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

A Anexo: Códigos

A.1. Configuración inicial

Código A.1: Importaciones, detección del entorno de ejecución y rutas de datos.

```
import os
import sys
import json
import numpy as np
import matplotlib.pyplot as plt
import PIL.Image as Image
import tensorflow as tf

IN_COLAB = 'google.colab' in sys.modules
IN_KAGGLE = os.path.exists('/kaggle/input')

if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    RUTA_DATOS = 'drive/MyDrive/TFG'
    RUTA_SALIDA = 'drive/MyDrive/TFG'
elif IN_KAGGLE:
    USUARIO_KAGGLE = 'jcnavarro0'
    NOMBRE_DATASET_KAGGLE = 'imagenestfg'
    RUTA_DATOS = os.path.join('/kaggle/input/datasets', USUARIO_KAGGLE,
                             NOMBRE_DATASET_KAGGLE) # solo lectura
    RUTA_SALIDA = '/kaggle/working'
else:
    RUTA_DATOS = '../..//IMAGENES_MEDICAS'
    RUTA_SALIDA = '../..//IMAGENES_MEDICAS'

print("Colab:", IN_COLAB, " Kaggle:", IN_KAGGLE)
print("TensorFlow:", tf.__version__)
```

Código A.2: Rutas de las carpetas de datos, configuración global del experimento y función de carga y pre-procesado (recorte y redimensionado) de una imagen.

```
RUTA_AUMENTADOS = os.path.join(RUTA_DATOS, 'datos_aumentados')
RUTA_RETICULO = os.path.join(RUTA_DATOS, 'datos_test_reticulo')
RUTA_SIN_PATRON = os.path.join(RUTA_DATOS, 'datos_test_sinPatron')
RUTA_BCC = os.path.join(RUTA_DATOS, 'BCC_ISIC')
RUTA_NOBCC = os.path.join(RUTA_DATOS, 'NoBCC_ISIC')
```

```

PATRONES = ['PigmentNetwork', 'Ulceration', 'Large_B_G_OvoidNests',
            'Multi_B_G_Globules', 'MapleLeaflike', 'SpokeWheel',
            'ArborizingTelangiectasia']

ABREVIATURAS_PATRONES = ['PN', 'ULC', 'OV', 'GL', 'ML', 'SW', 'AT']

IMG_SIZE = 256
UMBRAL_CLASIFICACION = 0.5
SEMILLA = 42
np.random.seed(SEMILLA)

# Cuantas capas finales de VGG16 se descongelan:
NUM_CAPAS_DESCONGELADAS = 5

INCLUIR_RETICULO = True
INCLUIR_SINPATRON = True

APLICAR_RECORTE_RETICULO = True
APLICAR_RECORTE_SINPATRON = True

PORCENTAJE_RECORTE = 0.6

if INCLUIR_RETICULO:
    _sufijo_reticulo = 'conReticulo' + ('Recorte' if APLICAR_RECORTE_RETICULO else
    'SinRecorte')
else:
    _sufijo_reticulo = 'sinReticulo'

if INCLUIR_SINPATRON:
    _sufijo_sinpatron = 'conSinPatron' + ('Recorte' if APLICAR_RECORTE_SINPATRON
    else 'SinRecorte')
else:
    _sufijo_sinpatron = 'sinSinPatron'

_SUFIJO_EXPERIMENTO = _sufijo_reticulo + '_' + _sufijo_sinpatron + '_ultimas5capas'

RUTA_MODELO = os.path.join(RUTA_SALIDA, 'modelos_keras', 'TFG_JCNG_modelo_' +
    _SUFIJO_EXPERIMENTO + '.keras')
RUTA_HISTORIA = os.path.join(RUTA_SALIDA, 'modelos_keras', 'TFG_JCNG_historia_' +
    _SUFIJO_EXPERIMENTO + '.json')
RUTA_TEST_DATOS = os.path.join(RUTA_SALIDA, 'modelos_keras', 'TFG_JCNG_test_' +
    _SUFIJO_EXPERIMENTO + '.npz')
os.makedirs(os.path.dirname(RUTA_MODELO), exist_ok=True)

def cargar_imagen(ruta):
    imagen = Image.open(ruta)
    imagen = imagen.convert('RGB')

    if RUTA_AUMENTADOS not in ruta:
        if (RUTA_BCC in ruta) or (RUTA_NOBCC in ruta):
            aplicar_recorte = APLICAR_RECORTE_BCCNOBCC
            porcentaje_recorte = PORCENTAJE_RECORTE_BCCNOBCC
        elif RUTA_RETICULO in ruta:
            aplicar_recorte = APLICAR_RECORTE_RETICULO
            porcentaje_recorte = PORCENTAJE_RECORTE
        else:
            aplicar_recorte = APLICAR_RECORTE_SINPATRON
            porcentaje_recorte = PORCENTAJE_RECORTE

    if aplicar_recorte:

```

```

        ancho, alto = imagen.size
        ancho_recorte = int(ancho * porcentaje_recorte)
        alto_recorte = int(alto * porcentaje_recorte)
        izquierda = (ancho - ancho_recorte) // 2
        arriba = (alto - alto_recorte) // 2
        imagen = imagen.crop((izquierda, arriba, izquierda + ancho_recorte,
                               arriba + alto_recorte))

    if imagen.size != (IMG_SIZE, IMG_SIZE):
        imagen = imagen.resize((IMG_SIZE, IMG_SIZE))
    return np.asarray(imagen)

```

A.2. Parte A: entrenamiento del modelo

Código A.3: Construcción del conjunto de imágenes de entrenamiento y reparto en entrenamiento, validación y test.

```

rutas_totales = []
etiquetas_totales = []

# 1) datos_aumentados: la etiqueta esta en el nombre del fichero
ficheros_aumentados = os.listdir(RUTA_AUMENTADOS)
for nombre in ficheros_aumentados:
    etiqueta = [0, 0, 0, 0, 0, 0, 0, 0]
    for i in range(len(PATRONES)):
        if PATRONES[i] in nombre:
            etiqueta[i] = 1
    rutas_totales.append(os.path.join(RUTA_AUMENTADOS, nombre))
    etiquetas_totales.append(etiqueta)

print("Imágenes de datos_aumentados:", len(ficheros_aumentados))

if INCLUIR_RETICULO:
    # 2) datos_test_reticulo: todas tienen patron reticular (PigmentNetwork = 1)
    ficheros_reticulo = os.listdir(RUTA_RETICULO)
    for nombre in ficheros_reticulo:
        etiqueta = [1, 0, 0, 0, 0, 0, 0, 0]
        rutas_totales.append(os.path.join(RUTA_RETICULO, nombre))
        etiquetas_totales.append(etiqueta)

    print("Imágenes de datos_test_reticulo:", len(ficheros_reticulo))
else:
    print("INCLUIR_RETICULO = False -> no se anade datos_test_reticulo al
          entrenamiento")

if INCLUIR_SINPATRON:
    # 3) datos_test_sinPatron: no tienen ningun patron BCC
    ficheros_sin_patron = os.listdir(RUTA_SIN_PATRON)
    for nombre in ficheros_sin_patron:
        etiqueta = [0, 0, 0, 0, 0, 0, 0, 0]
        rutas_totales.append(os.path.join(RUTA_SIN_PATRON, nombre))
        etiquetas_totales.append(etiqueta)

    print("Imágenes de datos_test_sinPatron:", len(ficheros_sin_patron))
else:
    print("INCLUIR_SINPATRON = False -> no se anade datos_test_sinPatron al
          entrenamiento")

print("Total de imágenes:", len(rutas_totales))

```

```

datos_completos = list(zip(rutas_totales, etiquetas_totales))
np.random.shuffle(datos_completos)

total_imagenes = len(datos_completos)
num_train = round(0.7 * total_imagenes)
num_val = round(0.2 * total_imagenes)

lista_train = datos_completos[0:num_train]
lista_val = datos_completos[num_train:num_train + num_val]
lista_test = datos_completos[num_train + num_val:]

print("Train:", len(lista_train))
print("Val:", len(lista_val))
print("Test:", len(lista_test))

```

Código A.4: Carga en memoria de las imágenes de entrenamiento, validación y test.

```

def cargar_lista_de_imagenes(lista):
    x = []
    y = []
    for ruta, etiqueta in lista:
        imagen = cargar_imagen(ruta)
        x.append(imagen)
        y.append(etiqueta)
    x = np.asarray(x)
    y = np.asarray(y)
    return x, y

print("Cargando imagenes de entrenamiento... (puede tardar varios minutos)")
train_set_x, train_set_y = cargar_lista_de_imagenes(lista_train)

print("Cargando imagenes de validacion...")
val_set_x, val_set_y = cargar_lista_de_imagenes(lista_val)

print("Cargando imagenes de test...")
test_set_x, test_set_y = cargar_lista_de_imagenes(lista_test)

print("Formas -> train_x:", train_set_x.shape, "val_x:", val_set_x.shape, "test_x:",
      test_set_x.shape)

```

Código A.5: Construcción del modelo VGG16 con ajuste fino de las últimas capas, cabeza clasificadora y compilación.

```

input_shape = (IMG_SIZE, IMG_SIZE, 3)
vgg = tf.keras.applications.vgg16.VGG16(include_top=False, weights='imagenet',
    input_shape=input_shape)

for capa in vgg.layers:
    capa.trainable = False
for capa in vgg.layers[-NUM_CAPAS_DESCONGELADAS:]:
    capa.trainable = True

x = tf.keras.layers.GlobalAveragePooling2D()(vgg.output)
x = tf.keras.layers.Dense(512, activation='relu')(x)
x = tf.keras.layers.Dropout(0.15)(x)
salida = tf.keras.layers.Dense(len(PATRONES), activation='sigmoid')(x)
modelo_completo = tf.keras.models.Model(inputs=vgg.input, outputs=salida)

```

```

optimizador = tf.keras.optimizers.Adam(learning_rate=0.00001, beta_1=0.9, beta_2
    =0.999, epsilon=1e-8, amsgrad=True)
modelo_completo.compile(loss='binary_crossentropy', optimizer=optimizador, metrics
   =['binary_accuracy'])
modelo_completo.summary()

```

Código A.6: Preprocesado final de las imágenes y entrenamiento del modelo completo.

```

train_set_x_prep = tf.keras.applications.vgg16.preprocess_input(train_set_x.astype
    ('float32'))
val_set_x_prep = tf.keras.applications.vgg16.preprocess_input(val_set_x.astype('
    float32'))
test_set_x_prep = tf.keras.applications.vgg16.preprocess_input(test_set_x.astype('
    float32'))

EPOCHS = 20
BATCH_SIZE = 32

print("*Comienza el entrenamiento*")
historia = modelo_completo.fit(
    x=train_set_x_prep, y=train_set_y,
    epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1,
    validation_data=(val_set_x_prep, val_set_y),
    shuffle=True,
)

modelo_completo.save(RUTA_MODELO)
with open(RUTA_HISTORIA, 'w') as f:
    json.dump(historia.history, f)

np.savez(RUTA_TEST_DATOS, test_set_x=test_set_x, test_set_y=test_set_y)

print("Modelo guardado en:", RUTA_MODELO)

```

Código A.7: Curvas de entrenamiento: evolución del *binary accuracy* y de la función de pérdida.

```

if 'historia' in dir():
    datos_historia = historia.history
else:
    with open(RUTA_HISTORIA, 'r') as f:
        datos_historia = json.load(f)

acc = datos_historia['binary_accuracy']
val_acc = datos_historia['val_binary_accuracy']
loss = datos_historia['loss']
val_loss = datos_historia['val_loss']
epochs_range = range(len(acc))

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Entrenamiento')
plt.plot(epochs_range, val_acc, label='Validación')
plt.legend(loc='lower right')
plt.title('Binary Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Entrenamiento')
plt.plot(epochs_range, val_loss, label='Validación')
plt.legend(loc='upper right')
plt.title('Loss')

```

```
plt.show()
```

A.3. Parte B: evaluación del modelo

Código A.8: Carga del modelo entrenado y de los datos de test guardados, sin necesidad de repetir el entrenamiento.

```
if not os.path.exists(RUTA_MODELO) or not os.path.exists(RUTA_TEST_DATOS):
    print("No se encuentra el modelo o los datos de test guardados.")
else:
    modelo_completo = tf.keras.models.load_model(RUTA_MODELO)

    datos_test_guardados = np.load(RUTA_TEST_DATOS)
    test_set_x = datos_test_guardados['test_set_x']
    test_set_y = datos_test_guardados['test_set_y']
    test_set_x_prep = tf.keras.applications.vgg16.preprocess_input(test_set_x.
                                                                    astype('float32'))

    print("Modelo y datos de test cargados correctamente desde", RUTA_SALIDA)
```

Código A.9: Clasificación del test interno mediante umbral y cálculo de sensibilidad y especificidad por patrón.

```
probs_test = modelo_completo.predict(test_set_x_prep)
pred_test = probs_test >= UMBRAL_CLASIFICACION

matrices_confusion_patrones = []

for i in range(len(PATRONES)):
    patron = PATRONES[i]
    y_true = test_set_y[:, i]
    y_pred = pred_test[:, i]

    tp = 0
    fp = 0
    tn = 0
    fn = 0
    for j in range(len(y_true)):
        if y_pred[j] == 1 and y_true[j] == 1:
            tp = tp + 1
        elif y_pred[j] == 1 and y_true[j] == 0:
            fp = fp + 1
        elif y_pred[j] == 0 and y_true[j] == 0:
            tn = tn + 1
        elif y_pred[j] == 0 and y_true[j] == 1:
            fn = fn + 1

    matrices_confusion_patrones.append([tp, fp, tn, fn])

    if (tp + fn) > 0:
        sensibilidad = tp / (tp + fn)
    else:
        sensibilidad = None

    if (tn + fp) > 0:
        especificidad = tn / (tn + fp)
    else:
        especificidad = None
```

```

print(patron)
print(" TP =", tp, " FP =", fp, " TN =", tn, " FN =", fn)
print(" Sensibilidad =", sensibilidad)
print(" Especificidad =", especificidad)
print("")

```

Código A.10: Matrices de confusión individuales para cada patrón dermatoscópico (test interno).

```

plt.figure(figsize=(16, 8))

for i in range(len(PATRONES)):
    tp, fp, tn, fn = matrices_confusion_patrones[i]
    matriz = [[tp, fn], [fp, tn]]

    plt.subplot(2, 4, i + 1)
    plt.imshow(matriz, cmap='Blues')
    plt.xticks([0, 1], ['Presente', 'Ausente'])
    plt.yticks([0, 1], ['Presente', 'Ausente'])
    plt.xlabel('Prediccion')
    plt.ylabel('Real')
    plt.title(PATRONES[i], fontsize=10)

    for fila in range(2):
        for columna in range(2):
            plt.text(columna, fila, str(matriz[fila][columna]), ha='center', va='center',
                    fontsize=12)

plt.tight_layout()
plt.show()

```

Código A.11: Evaluación sobre el conjunto externo BCC_ISIC/NoBCC_ISIC.

```

APLICAR_RECORTE_BCCNOBCC = True
PORCENTAJE_RECORTE_BCCNOBCC = 0.6

ficheros_bcc = os.listdir(RUTA_BCC)
ficheros_nobcc = os.listdir(RUTA_NOBCC)

imagenes_bccnobcc = []
nombres_bccnobcc = []
clases_bccnobcc = [] # clase clinica: BCC o No-BCC, segun la carpeta de origen
es_bcc_de_verdad = [] # 1 si la imagen es realmente BCC, 0 si no

for nombre in ficheros_bcc:
    ruta = os.path.join(RUTA_BCC, nombre)
    imagenes_bccnobcc.append(cargar_imagen(ruta))
    nombres_bccnobcc.append(nombre)
    clases_bccnobcc.append('BCC')
    es_bcc_de_verdad.append(1)

for nombre in ficheros_nobcc:
    ruta = os.path.join(RUTA_NOBCC, nombre)
    imagenes_bccnobcc.append(cargar_imagen(ruta))
    nombres_bccnobcc.append(nombre)
    clases_bccnobcc.append('No-BCC')
    es_bcc_de_verdad.append(0)

```

```

print("Cargadas", len(ficheros_bcc), "imagenes de BCC y", len(ficheros_nobcc), "de
      NoBCC")

x_bccnobcc = np.asarray(imagenes_bccnobcc)
x_bccnobcc_prep = tf.keras.applications.vgg16.preprocess_input(x_bccnobcc.astype('
      float32'))
probs_bccnobcc = modelo_completo.predict(x_bccnobcc_prep)
pred_patrones_bccnobcc = probs_bccnobcc >= UMBRAL_CLASIFICACION

indice_pigment_network = 0

tp = 0
fp = 0
tn = 0
fn = 0

for i in range(len(nombres_bccnobcc)):
    tiene_reticular = pred_patrones_bccnobcc[i][indice_pigment_network]

    tiene_algun_otro_patron = False
    for j in range(1, len(PATRONES)):
        if pred_patrones_bccnobcc[i][j] == 1:
            tiene_algun_otro_patron = True

    # Criterio clinico: BCC = sin patron reticular
    # + al menos uno de los otros 6 patrones presente.
    if (not tiene_reticular) and tiene_algun_otro_patron:
        prediccion_bcc = 1
    else:
        prediccion_bcc = 0

    verdad = es_bcc_de_verdad[i]

    if prediccion_bcc == 1 and verdad == 1:
        tp = tp + 1
    elif prediccion_bcc == 1 and verdad == 0:
        fp = fp + 1
    elif prediccion_bcc == 0 and verdad == 0:
        tn = tn + 1
    elif prediccion_bcc == 0 and verdad == 1:
        fn = fn + 1

    patrones_disparados = ''
    for j in range(len(PATRONES)):
        if pred_patrones_bccnobcc[i][j] == 1:
            porcentaje = round(probs_bccnobcc[i][j] * 100)
            if patrones_disparados != '':
                patrones_disparados = patrones_disparados + ', '
            patrones_disparados = patrones_disparados + ABREVIATURAS_PATRONES[j] + ' (
                ' + str(porcentaje) + '%)'

    if prediccion_bcc != verdad:
        print("Fallo en la imagen:", nombres_bccnobcc[i], " (clase real:",
              clases_bccnobcc[i], ") patrones:", patrones_disparados)
    else:
        print("Acierto en la imagen:", nombres_bccnobcc[i], " (clase real:",
              clases_bccnobcc[i], ") patrones:", patrones_disparados)

print("")
print("TP =", tp, " FP =", fp, " TN =", tn, " FN =", fn)
print("Accuracy =", (tp + tn) / (tp + fp + tn + fn))
if (tp + fn) > 0:
    print("Sensibilidad =", tp / (tp + fn))

```

```

if (tn + fp) > 0:
    print("Especificidad =", tn / (tn + fp))
if (tp + fp) > 0:
    print("PPV =", tp / (tp + fp))

```

Código A.12: Matriz de confusión 2x2 del criterio clínico BCC/No-BCC, con umbral 0.5.

```

# Matriz de confusion 2x2 del criterio clinico BCC/No-BCC (umbral 0.5, Seccion 6)
matriz_confusion = [[tp, fn], [fp, tn]]

plt.figure(figsize=(5, 5))
plt.imshow(matriz_confusion, cmap='Blues')
plt.xticks([0, 1], ['BCC', 'No-BCC'])
plt.yticks([0, 1], ['BCC', 'No-BCC'])
plt.xlabel('Prediccion del modelo')
plt.ylabel('Clase real')
plt.title('Matriz de confusion BCC/No-BCC')

for fila in range(2):
    for columna in range(2):
        plt.text(columna, fila, str(matriz_confusion[fila][columna]), ha='center',
            va='center', fontsize=16)

plt.tight_layout()
plt.show()

```

A.4. Parte C: mapas de activación (Grad-CAM)

Código A.13: Definición del modelo intermedio de Grad-CAM y de las funciones para calcular y mostrar el mapa de calor.

```

CAPA_GRAD_CAM = 'block5_pool'

capa_conv_grad_cam = modelo_completo.get_layer(CAPA_GRAD_CAM)
modelo_grad_cam = tf.keras.models.Model(
    inputs=modelo_completo.input,
    outputs=[capa_conv_grad_cam.output, modelo_completo.output],
)

def hacer_grad_cam(imagen, indice_patron):
    imagen_prep = tf.keras.applications.vgg16.preprocess_input(imagen.astype('float32'))
    entrada = np.expand_dims(imagen_prep, axis=0)

    with tf.GradientTape() as cinta:
        mapas_conv, predicciones = modelo_grad_cam(entrada)
        salida_patron = predicciones[:, indice_patron]

    # Gradiente de la salida del patron respecto a los mapas de activacion de VGG16 :
    # dice cuanto cambiaria la prediccion si cambiara un poco cada filtro.
    gradientes = cinta.gradient(salida_patron, mapas_conv)

    # Importancia de cada uno de los filtros: la media de su gradiente
    peso_por_filtro = tf.reduce_mean(gradientes[0], axis=(0, 1)).numpy()

```

```

mapas_conv = mapas_conv[0].numpy()
alto_mapa, ancho_mapa, num_filtros = mapas_conv.shape

mapa_calor = np.zeros((alto_mapa, ancho_mapa))
for filtro in range(num_filtros):
    mapa_calor = mapa_calor + peso_por_filtro[filtro] * mapas_conv[:, :, filtro]

# Solo interesa lo que suma a favor del patron (ReLU), y normalizamos a [0, 1]
mapa_calor = np.maximum(mapa_calor, 0)
if mapa_calor.max() > 0:
    mapa_calor = mapa_calor / mapa_calor.max()

return mapa_calor

def mostrar_grad_cam(imagen, mapa_calor, titulo):
    imagen_calor_grande = Image.fromarray(np.uint8(mapa_calor * 255))
    imagen_calor_grande = imagen_calor_grande.resize((IMG_SIZE, IMG_SIZE))
    mapa_calor_grande = np.asarray(imagen_calor_grande)

    plt.figure(figsize=(9, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(imagen)
    plt.title('Imagen original')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(imagen)
    plt.imshow(mapa_calor_grande, cmap='jet', alpha=0.5)
    plt.title(titulo, fontsize=9)
    plt.axis('off')

    plt.tight_layout()
    plt.show()

```

Código A.14: Recorrido de las imágenes BCC/No-BCC con Grad-CAM, mostrando únicamente las que activan un patrón concreto.

```

PATRON_A_VISUALIZAR = 'ArborizingTelangiectasia'

indice_patron_visualizar = PATRONES.index(PATRON_A_VISUALIZAR)

# Solo nos quedamos con las imagenes donde el patron esta activado
indices_activados = []
for i in range(len(nombres_bccnobcc)):
    if pred_patrones_bccnobcc[i][indice_patron_visualizar] == 1:
        indices_activados.append(i)

print("Patron:", PATRON_A_VISUALIZAR)
print("Activado en", len(indices_activados), "de", len(nombres_bccnobcc), "imagenes")
print("")

for i in indices_activados:
    probabilidad = round(probs_bccnobcc[i][indice_patron_visualizar], 2)

    if es_bcc_de_verdad[i] == 1:
        origen = 'BCC'
    else:

```

```
origen = 'NoBCC'

titulo = PATRON_A_VISUALIZAR + ' (' + nombres_bccnobcc[i] + ', ' + origen + ', '
        clase: ' + clases_bccnobcc[i] + ', prob: ' + str(probabilidad) + '))'
print(titulo)

mapa_calor = hacer_grad_cam(imagenes_bccnobcc[i], indice_patron_visualizar)
mostrar_grad_cam(imagenes_bccnobcc[i], mapa_calor, titulo)
```